

Sélection des tests de non-régression (RTS)

HAI823I - TER M1

Master Génie Logiciel

Université de Montpellier

Étudiants :

Sami CHBICHEB

Samir EL KHOUMRI

Aly HACHEM REDA

Encadrant :

Abdelhak Djamel Seriai

Résumé

Ce projet s’inscrit dans la problématique de la **sélection de tests de non-régression** (*Regression Test Selection*, RTS) appliquée aux projets utilisant le *Behavior-Driven Development* (BDD) et le framework Cucumber/Gherkin. Dans ce contexte, chaque modification du code source soulève la question suivante : quels tests faut-il ré-exécuter pour détecter d’éventuelles régressions, sans pour autant rejouer l’intégralité de la suite de tests ?

Notre mission a consisté à concevoir et implémenter plusieurs outils de sélection de tests complémentaires, en réponse aux limitations des approches existantes qui traitent le runner Cucumber comme un bloc monolithique et sont incapables d’identifier les scénarios BDD individuellement impactés.

Trois outils ont été développés :

- un outil de sélection des **tests unitaires**, exploitant l’analyse statique du code source Java ;
- un outil **BDD boîte noire**, fondé sur la similarité sémantique entre scénarios Gherkin via une vectorisation TF-IDF et une détection de seuil par knee point, permettant de relier chaque scénario d’acceptation à ses tests unitaires sous-jacents ;
- un outil **BDD via LLM**, exploitant la capacité de raisonnement des grands modèles de langage (OpenAI, Gemini, Claude, Ollama) pour analyser un `git diff` et identifier les scénarios fonctionnellement impactés par les modifications.

Ce mémoire présente l’état de l’art conduit sur la littérature scientifique relative à la RTS, les choix méthodologiques et techniques retenus, les développements réalisés, ainsi qu’une réflexion sur la validation expérimentale et les perspectives d’amélioration de ces outils.

Remerciements

Nous souhaitons adresser nos sincères remerciements à **M. Abdelhak Djamel Seriai** pour son encadrement attentif et sa disponibilité tout au long de ce projet. Ses orientations bibliographiques, ses retours réguliers sur nos travaux et sa capacité à nous guider face aux difficultés techniques ont été déterminants dans la progression de ce TER.

Nous le remercions également pour la liberté qu’il nous a accordée dans l’exploration des approches, tout en maintenant un cadre rigoureux propice à la réflexion scientifique. Ses conseils nous ont permis de prendre du recul sur nos choix de conception et d’approfondir notre compréhension du domaine de la qualité logicielle et des tests de non-régression.

Table des matières

1	Introduction	5
1.1	Problématique	5
1.2	Objectifs	5
2	Le domaine des tests	6
2.1	Niveaux de granularité	6
2.2	Techniques de tests	7
2.2.1	Black-Box	7
2.2.2	White-Box	7
2.2.3	Grey-Box	8
2.3	Tests de non-régression	8
2.4	Tests fonctionnels et Behavior-Driven Development	9
3	Etude de l'état de l'art	10
3.1	Contexte : tests et non-régression	10
3.2	Protocole de recherche	10
3.3	Quelles approches de RTS existent ?	11
3.4	Quel niveau de granularité ?	12
3.5	Les approches sont-elles sûres ?	12
3.6	Quels algorithmes et outils ?	13
3.7	Quels langages et quelles métriques d'évaluation ?	13
3.8	RTS et BDD : état des lieux	14
3.9	Synthèse : matrice de comparaison	15
4	Approches Scientifiques	16
4.1	Approche : RTS pour tests unitaires par analyse statique de dépendances	16
4.2	Approche : RTS-BDD par similarité sémantique	16
4.3	RTS BDD via LLM	17
4.3.1	pourquoi les LLM sont adaptés au problème	18
4.3.2	L'approche hybride : analyse statique et raisonnement LLM	18
5	Implémentation	20
5.1	RTS sur les tests unitaires	20
5.1.1	Intérêt de l'outil	20

5.1.2	Fonctionnement de l'outil	20
5.1.3	Architecture logicielle	21
5.1.4	Résultat et utilisation	22
5.1.5	Limitations et transition vers les approches BDD	22
5.2	BDD black box	23
5.2.1	Intérêt de l'outil	23
5.2.2	Fonctionnement de l'outil	23
5.2.3	Parsing des scénarios existants	25
5.2.4	Prétraitement du texte	26
5.2.5	Vectorisation TF-IDF	26
5.2.6	Calcul de la similarité cosinus	27
5.2.7	Détection du knee point	27
5.2.8	Mapping AT/UT	28
5.2.9	Résultat et utilisation	28
5.2.10	Limitations	29
5.3	BDD LLM	30
5.3.1	Intérêt de l'outil	30
5.3.2	Première version : sélection par le LLM seul	30
5.3.3	Seconde version : analyse statique en amont du LLM	31
6	Méthodologie de Validation et Vérité terrain	34
6.1	Le problème de l'évaluation	34
6.2	Définition de la vérité terrain	34
6.2.1	Principe	34
6.2.2	Couverture dynamique avec JaCoCo	34
6.3	Métriques d'évaluation	35
6.4	Mise en place concrète	36
6.4.1	Collecte de la couverture par scénario	36
6.4.2	Calcul de la vérité terrain pour un commit	36
6.4.3	Application sur l'historique Git	36
6.5	Limites connues de la vérité terrain	37
7	Résultats et Benchmarks	38
7.1	Résultats quantitatifs	38
7.1.1	Métriques moyennes	38

7.1.2	Distribution du rappel par commit	39
7.2	Interprétation des résultats	39
7.2.1	Pourquoi le mode hybride obtient une précision bien meilleure	39
7.2.2	Pourquoi le rappel n'augmente presque pas	40
7.2.3	Lecture synthétique : deux profils de commits	41
7.3	intégration CI/CD	41
7.4	Intégration des API LLM distantes	41
7.5	Perspectives d'amélioration	42
8	Gestion de projet	43
8.1	Déroulement du projet	43
8.2	Organisation et gestion de projet	43
8.3	Réunions hebdomadaires avec l'encadrant	43
8.4	Objectifs expérimentaux	44
9	Bilan	45

1 Introduction

1.1 Problématique

La qualité d'un logiciel se mesure à sa capacité à répondre aux besoins du client tout au long de son cycle de vie. Le principal levier pour garantir cette qualité est le test. Il permet de vérifier, à différents niveaux de granularité (unitaire, intégration, système, acceptation) que le logiciel se comporte comme attendu. Loin d'être une étape ponctuelle, le test accompagne toute la vie du projet, car chaque évolution du code peut introduire de nouveaux défauts.

C'est là qu'interviennent les tests de non-régression : à chaque modification, il faut s'assurer qu'aucune fonctionnalité existante n'a été dégradée. En pratique, cela revient souvent à rejouer l'intégralité de la suite de tests à chaque commit. Un coût qui devient rapidement prohibitif sur un projet réel, où l'exécution peut représenter plusieurs dizaines de minutes, voire des heures. La sélection de tests de non-régression (Regression Test Selection, RTS) répond à ce problème en ne ré-exécutant que les tests réellement susceptibles d'être affectés par la modification.

Bien établie pour les tests classiques, la RTS reste pourtant quasi inexplorée dans le contexte du Behavior-Driven Development (BDD), où les exigences sont exprimées sous forme de scénarios en langage naturel structuré (Gherkin). Les outils existants comme Ekstazi ou STARTS raisonnent au niveau des classes Java et ne savent pas distinguer quel scénario est touché par un changement. Le choix se réduit alors à tout rejouer ou ne rien rejouer. C'est cette problématique que le TER cherche à résoudre, autour d'une question centrale : étant donné une modification du code source, quels scénarios BDD est-il nécessaire et suffisant de ré-exécuter pour détecter d'éventuelles régressions ?

1.2 Objectifs

Ce TER s'est fixé quatre objectifs complémentaires :

1. **Cartographier l'état de l'art** : identifier les grandes familles d'approches RTS existantes et confirmer le manque de littérature sur le RTS appliqué au BDD.
2. **Concevoir des approches adaptées au BDD** : proposer des méthodes capables de sélectionner des scénarios Gherkin individuels (analyse statique, similarité sémantique, raisonnement LLM).
3. **Implémenter ces approches** : produire des outils concrets, testés et intégrables dans un pipeline Java/Maven sans modification du code source cible.
4. **Évaluer les outils** : construire une vérité terrain par instrumentation dynamique (JaCoCo) et mesurer la précision de chaque outil sur un historique réel de commits.

2 Le domaine des tests

2.1 Niveaux de granularité

Les tests s'organisent selon quatre niveaux de granularité, du plus fin au plus global, chacun ciblant une portée différente du système.

Tests unitaires

Les tests unitaires constituent la base des pratiques modernes de développement logiciel. Ils visent à vérifier la correction d'unités de code individuelles (comme des méthodes ou des fonctions) de manière isolée. Ce sont les tests les plus nombreux et les plus rapides à exécuter.

Tests d'intégration

Les tests d'intégration interviennent après les tests unitaires et ont pour objectif de vérifier les interactions entre les différents modules ou composants d'un système. Alors que le test unitaire isole une fonction, le test d'intégration s'assure que l'assemblage de ces unités produit le comportement attendu.

Tests système

Les tests système interviennent après les tests d'intégration et évaluent le comportement d'un système logiciel complet et entièrement intégré. Ce niveau de test vérifie que le système satisfait les exigences spécifiées, tant fonctionnelles que non fonctionnelles.

Tests d'acceptation

Les tests d'acceptation constituent la dernière phase du cycle de test avant la mise en production. Ils visent à valider que le logiciel répond aux besoins métier et aux attentes des utilisateurs finaux, plutôt qu'à vérifier des aspects purement techniques.

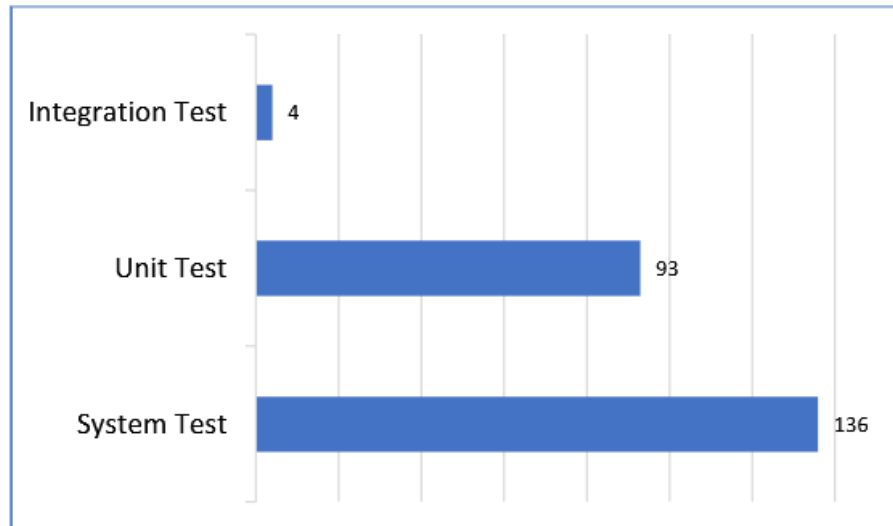


FIGURE 1 – Sondage sur les niveaux de tests les plus utilisés auprès d'entreprises

2.2 Techniques de tests

Les techniques de tests se répartissent en trois catégories selon le degré de connaissance de la structure interne du logiciel : la boîte noire (black-box), la boîte blanche (white-box) et la boîte grise (grey-box).

2.2.1 Black-Box

La technique de la boîte noire consiste à analyser le comportement du logiciel sans prendre en compte la structure interne du code : la sortie est observée selon différents paramètres d'entrée. Plusieurs techniques formelles en boîte noire existent, comme le partitionnement en classes d'équivalence ou l'analyse des valeurs limites.

2.2.2 White-Box

Contrairement aux techniques black-box, les techniques en boîte blanche sont conçues en ayant connaissance de la logique interne du logiciel. Le programme est alors représenté sous la forme d'un graphe de flot de contrôle, l'objectif étant de maximiser la couverture de code. Pour cela, plusieurs techniques sont disponibles, notamment la couverture d'instructions, la couverture de branches ou la couverture de chemins.

2.2.3 Grey-Box

La technique de la boîte grise est une approche hybride combinant les deux précédentes : le testeur dispose d’une connaissance partielle de la structure interne du logiciel, qu’il exploite pour concevoir des tests évalués ensuite sur leur comportement externe. Elle vise à tirer parti des avantages des deux approches.

Ces différentes techniques sont au cœur de notre travail : nos outils s’appuient tantôt sur une analyse boîte noire des scénarios fonctionnels, tantôt sur une analyse boîte blanche du code source. Cette dernière est par ailleurs cruciale pour la mise en place des tests de régression, que nous verrons plus en détail ultérieurement.

2.3 Tests de non-régression

Le développement d’un logiciel ne s’arrête pas à sa première mise en production. Il évolue continuellement à travers l’ajout de fonctionnalités, ou de corrections de bugs. Or, toute modification du code, même mineure, peut involontairement altérer un comportement qui fonctionnait jusque-là correctement. On parle alors de régression.

Les tests de non-régression visent précisément à détecter ces régressions. Après chaque modification, on ré-exécute des tests existants afin de s’assurer que les fonctionnalités déjà validées continuent de se comporter comme attendu. En pratique, l’approche la plus simple consiste à rejouer l’intégralité de la suite de tests à chaque modification. Cette stratégie, dite retest-all, garantit une couverture maximale mais devient rapidement prohibitive : sur un projet de taille réelle, l’exécution complète peut représenter plusieurs dizaines de minutes, voire plusieurs heures, ce qui est incompatible avec un rythme d’intégration continue où les commits sont fréquents.

Pour réduire ce coût sans sacrifier la capacité à détecter les régressions, la littérature distingue trois grandes stratégies complémentaires :

- **Minimisation** : réduction de la suite de tests par élimination des cas redondants, afin de conserver un ensemble plus compact couvrant les mêmes exigences.
- **Priorisation** (*Test Case Prioritization*, TCP) : ordonnancement des tests selon un critère de criticité, de manière à détecter les fautes au plus tôt dans l’exécution de la suite.
- **Sélection** (*Regression Test Selection*, RTS) : sélection du seul sous-ensemble de tests susceptibles d’être affectés par la modification considérée, les autres étant écartés de la ré-exécution.

C’est cette dernière stratégie, la sélection (RTS), qui constitue le cœur de notre travail : son objectif est de réduire significativement le temps d’exécution de la suite de tests tout en préservant la capacité à détecter les régressions.

2.4 Tests fonctionnels et Behavior-Driven Development

Parmi les tests présentés précédemment, on distingue les tests *fonctionnels*, qui vérifient *ce que* fait le logiciel — sa conformité aux exigences métier — par opposition aux tests non fonctionnels qui s'intéressent à *comment* il le fait (performance, sécurité, robustesse...). Les tests fonctionnels raisonnent donc en termes de comportement observable : pour une entrée donnée, le système produit-il la sortie attendue ?

Le *Behavior-Driven Development* (BDD) pousse cette logique à son terme en plaçant le comportement attendu au centre du processus de développement. Plutôt que de décrire les exigences dans des documents séparés du code, le BDD les exprime sous forme de *scénarios* rédigés dans un langage naturel structuré, compréhensible aussi bien par les développeurs que par les parties prenantes non techniques (clients, analystes métier).

Ces scénarios sont écrits en **Gherkin**, une syntaxe reposant sur quelques mots-clés structurants :

- **Given** (Étant donné) : le contexte initial, l'état du système avant l'action ;
- **When** (Quand) : l'action ou l'événement déclencheur ;
- **Then** (Alors) : le résultat attendu, le comportement à vérifier.

Voici un exemple de scénario Gherkin :

```
1 Scenario: Computing the modulo of two integers
2   Given an integer operation '%'
3   When I provide a first number 7
4   And I provide a second number 3
5   Then the operation evaluates to 1
```

Chaque étape (*step*) est reliée au code applicatif par une *step definition* : une méthode Java annotée (**@Given**, **@When**, **@Then**) qui implémente le comportement décrit. Le framework **Cucumber** assure le lien entre les deux : il lit les fichiers de scénarios (**.feature**), associe chaque étape à sa step definition correspondante, et exécute le tout comme un test fonctionnel automatisé.

L'intérêt du BDD pour notre travail tient à une propriété fondamentale : il établit, par construction, une *traçabilité* naturelle entre les exigences (scénarios Gherkin), leur implémentation (step definitions) et le code testé. Un même scénario constitue ainsi à la fois une spécification, une documentation et un test exécutable. C'est précisément cette double nature — fonctionnelle et exécutable — qui rend la sélection de tests de non-régression à la fois pertinente et délicate dans un contexte BDD, comme nous le verrons dans la suite de ce rapport.

3 Etude de l'état de l'art

Avant de concevoir nos propres outils, il était indispensable de comprendre ce qui existe déjà. Un état de l'art consiste à recenser, analyser et synthétiser les travaux scientifiques publiés sur un sujet donné, afin de dresser un panorama des approches existantes, d'en dégager les tendances et d'en identifier les limites. Cette démarche poursuit un double objectif : éviter de réinventer des solutions déjà éprouvées, et surtout repérer les manques de la littérature — autant d'opportunités sur lesquelles fonder une contribution originale.

Dans notre cas, l'état de l'art vise à cartographier les approches de sélection de tests de non-régression (RTS) et à évaluer leur applicabilité au contexte BDD. La section suivante détaille le protocole de recherche suivi, avant de présenter et de comparer les approches retenues.

3.1 Contexte : tests et non-régression

Comme présenté en section 2.3, les tests de non-régression se déclinent en trois stratégies : minimisation, priorisation et sélection. Notre état de l'art se concentre sur la sélection (*Regression Test Selection*, RTS), dont l'objectif est de réduire le temps d'exécution de la suite de tests tout en préservant la capacité à détecter les régressions. Nous analysons ci-après les approches existantes selon plusieurs critères, avant de mettre en évidence le manque de travaux ciblant spécifiquement le contexte BDD.

3.2 Protocole de recherche

Notre étude bibliographique a été conduite selon un protocole structuré proche d'un *Systematic Mapping Study*. Nous avons interrogé la bibliothèque DBLP ainsi que l'outil Connected Papers à l'aide des mots-clés suivants : « regression test selection », « continuous integration testing » et « test case selection ».

Les critères d'inclusion retenus sont les suivants :

- appartenir au domaine du génie logiciel ;
- traiter la sélection de tests de régression (RTS) comme sujet principal ;
- ne pas se limiter à la génération de nouveaux cas de test ;
- proposer des évaluations expérimentales (pas d'articles purement théoriques) ;
- ne pas se concentrer sur un environnement ou un langage obsolète ou trop restreint.

Le filtrage a été mené en trois phases successives, dans un processus en entonnoir visant à maximiser le rappel (ne pas manquer d'articles pertinents) et la précision (ne garder que les articles réellement utiles) :

1. **Filtrage par abstract** : lecture des résumés pour éliminer les travaux hors domaine. Corpus réduit à 46 articles.
2. **Filtrage par introduction** : lecture des introductions pour affiner selon les critères d'inclusion. Corpus réduit à 22 articles.
3. **Lecture complète** : lecture intégrale pour ne retenir que les articles traitant directement la RTS de manière expérimentale. Corpus final : environ 15 articles.

Six critères d’analyse ont été définis pour comparer les approches : le type d’analyse (statique, dynamique, IR), le niveau de granularité, la sûreté, le type d’algorithme, le langage d’expérimentation, et les métriques d’évaluation. Ces critères correspondent directement aux questions de recherche développées ci-après.

3.3 Quelles approches de RTS existent ?

L’analyse du corpus fait apparaître quatre grandes familles d’approches.

Approches par analyse statique de graphes de dépendances. Ces méthodes construisent un graphe de dépendances entre entités du code source à partir d’une analyse syntaxique statique (AST), puis identifient les tests impactés par une modification via un parcours inversé de ce graphe. *Extraction-Based RTS* [6] opère au niveau fichier avec un très faible surcoût d’analyse (*overhead*). PKRTS [1] adopte le niveau package pour un meilleur équilibre précision/coût. *SemRTS* [7] enrichit cette démarche en distinguant les changements sémantiques (modification de comportement) des changements non sémantiques (renommage, reformatage), évitant ainsi de sélectionner des tests pour des modifications sans impact réel.

Approches dynamiques et évolutionnaires. Ces techniques exploitent l’historique d’exécution des tests pour guider la sélection. HyRTS [16] combine dynamiquement deux niveaux de granularité (méthode et fichier) pour maximiser la réduction. ARTeCS [3] propose un framework multi-objectif basé sur des algorithmes évolutionnaires, améliorant la détection de changements de 43% par rapport à l’état de l’art avec un temps d’exécution moyen de 9 secondes. L’approche par opérateur de mutation améliorée [14] affine l’opérateur de bit-flip classique pour favoriser les cas de test rentables. Le système XCSF [12], basé sur les *Learning Classifier Systems*, apprend des règles de sélection à partir de l’historique d’exécution.

Approches basées sur la recherche d’information (IR). Ces méthodes traitent les cas de test et les descriptions de modifications comme des documents textuels et calculent une similarité pour sélectionner les tests. AutoTestPlan [8] exploite Apache Lucene pour comparer des descriptions textuelles de bugs ou de changements avec le texte des cas de test, sans accès au code source. ReTEST [2] combine trois sources : l’historique de fautes, la diversité des cas de test et l’historique des changements. Ces approches sont intrinsèquement indépendantes du langage de programmation.

Approches basées sur la traçabilité (BDD). Une dernière catégorie exploite la traçabilité explicite entre exigences et tests, rendue naturelle par le BDD. L’approche COMPSAC 2021 [15] s’appuie sur les liens entre scénarios Gherkin et tests (acceptation + unitaires) pour sélectionner uniquement les tests impactés par une exigence modifiée. C’est la seule approche strictement RTS du corpus à cibler les projets BDD.

3.4 Quel niveau de granularité ?

Le niveau de granularité détermine l'unité à partir de laquelle les dépendances sont calculées. On distingue cinq niveaux principaux dans notre corpus.

Niveau méthode. Le plus fin des niveaux, il offre la meilleure précision mais implique un coût d'analyse élevé. HyRTS [16] adopte ce niveau quand les ressources le permettent.

Niveau fichier. Bon compromis entre précision et rapidité. C'est le choix de *Extraction-Based RTS* [6] et d'un des deux modes de HyRTS.

Niveau package. PKRTS [1] montre que ce niveau permet une réduction significative du temps d'analyse par rapport au niveau classe, tout en conservant une couverture satisfaisante.

Niveau test case. La majorité des approches ML, IR et évolutionnaires sélectionnent ou écartent chaque test dans son ensemble. C'est le cas de ARTeCS [3], ReTEST [2], AutoTestPlan [8], XCSF [12] et SemRTS [7].

Niveau scénario / fonctionnalité. Spécifique au BDD et aux systèmes configurables. COMPSAC 2021 [15] opère à double granularité : scénario (pour les tests d'acceptation) et test unitaire (via la traçabilité AT $\rightarrow$ UT). FeaTestSelPrio [9] opère au niveau de la fonctionnalité pour les systèmes hautement configurables.

Le choix du niveau de granularité est un arbitrage fondamental entre **précision** (moins de sur-sélection) et **coût d'analyse** (plus la granularité est fine, plus l'analyse est coûteuse).

3.5 Les approches sont-elles sûres ?

Une technique RTS est dite sûre (safe) si elle garantit de sélectionner au minimum tous les tests susceptibles de détecter une régression introduite par la modification considérée. C'est une propriété formelle : aucun test qui aurait pu échouer ne peut être omis.

Approches sûres. Les techniques basées sur l'analyse statique de graphes de dépendances sont généralement sûres car elles s'appuient sur des dépendances formellement vérifiables. C'est le cas de *Extraction-Based RTS* [6], PKRTS [1], SemRTS [7], HyRTS [16], ReTSE [10] (par exclusion du code inchangé), et l'approche *All-Uses* pour BPEL [5]. L'approche BDD de COMPSAC 2021 [15] est également sûre grâce à l'utilisation de liens de traçabilité explicites.

Approches non sûres (heuristiques). Les techniques basées sur l'apprentissage automatique, la recherche d'information ou les algorithmes évolutionnaires ne peuvent pas garantir l'exhaustivité de la sélection. C'est le cas de ARTeCS [3], XCSF [12], ReTEST [2], AutoTestPlan [8] et les approches de la revue ML [11]. Le bénéfice de ces approches réside dans des taux de réduction souvent plus élevés, au prix d'un risque maîtrisé de manquer certains tests défaillants.

Ce compromis sûreté/réduction est central : les contextes critiques (sécurité, médical) favoriseront les approches sûres, tandis que les environnements d'intégration continue à haute fréquence de commits privilégieront la réduction maximale.

3.6 Quels algorithmes et outils ?

Graphes de dépendances et AST. Plusieurs approches s'appuient sur un parseur syntaxique (JavaParser, etc.) pour construire un graphe de dépendances entre entités du code, mis à jour de façon incrémentale entre deux commits. *Extraction-Based RTS* [6], PKRTS [1] et SemRTS [7] illustrent cette famille. SemRTS y ajoute une analyse de la sémantique des changements via des règles sur l'AST.

Instrumentation dynamique. HyRTS [16] instrumente le code à l'exécution pour capturer les dépendances réelles entre tests et entités, évitant les faux positifs liés à des dépendances déclarées mais jamais exercées à l'exécution.

Recherche d'information : Apache Lucene. AutoTestPlan [8] et ReTEST [2] reposent sur des techniques d'IR. AutoTestPlan exploite directement le moteur Apache Lucene pour vectoriser les descriptions textuelles (TF-IDF) et calculer des scores de similarité. Ces approches sont rapides, légères et indépendantes du langage de programmation.

Algorithmes évolutionnaires. ARTeCS [3] et l'approche par mutation améliorée [14] utilisent des algorithmes génétiques multi-objectifs. L'opérateur de mutation guide l'évolution d'une population de tests vers un sous-ensemble optimal, en pondérant chaque cas de test selon son historique de détection de fautes et son coût d'exécution.

Apprentissage par renforcement : LCS. XCSF [12] est un *Learning Classifier System* qui apprend des règles de classification à partir de l'historique d'exécution (résultats passés, durée) pour attribuer une priorité à chaque test dans la limite d'un budget temporel donné.

Traçabilité BDD. L'approche COMPSAC 2021 [15] exploite la traçabilité naturelle des projets BDD : les liens entre scénarios Gherkin, step definitions annotées (@Given/@When/@Then) et tests unitaires permettent de propager mécaniquement l'impact d'une modification d'exigence sans analyser le code source.

3.7 Quels langages et quelles métriques d'évaluation ?

Langages d'expérimentation.

- **Java** : de loin le langage le plus utilisé ; HyRTS [16] (32 projets, 124 M LOC), ExtRTS [6], PKRTS [1], SemRTS [7] et ARTeCS [3].
- **C/C++** : utilisé par FeaTestSelPrio [9] pour les systèmes configurables.
- **C** : langage de ReTSE [10].
- **BPEL** (services composites) : utilisé par l'approche *All-Uses* [5].
- **Indépendant du langage** : les approches IR (AutoTestPlan, ReTEST) et ML (XCSF, revue SLR) n'analysent que des métadonnées textuelles ou comportementales.

Métriques d'évaluation. Les métriques employées pour mesurer la qualité des techniques RTS sont les suivantes :

- **Taux de réduction** (*reduction rate*) : pourcentage de tests non sélectionnés par rapport à la suite complète. Métrique principale pour quantifier le gain en temps d'exécution.
- **Taux de détection de fautes** (*fault detection rate*) : proportion de fautes détectées par la suite réduite par rapport à la suite complète.
- **APFD** (*Average Percentage of Fault Detection*) : métrique de priorisation mesurant la rapidité de détection des fautes. Utilisée par XCSF [12] et les approches ML.
- **Rappel et précision** : le rappel mesure si tous les tests pertinents ont bien été sélectionnés ; la précision mesure si les tests sélectionnés sont bien pertinents. Utilisées par AutoTestPlan [8] et l'approche *All-Uses* [5].
- **Surcoût d'analyse** (*overhead*) : temps additionnel requis pour l'analyse RTS elle-même. Particulièrement étudié par ExtRTS [6] et HyRTS [16].

3.8 RTS et BDD : état des lieux

Le *Behavior-Driven Development* (BDD) est un paradigme dans lequel les exigences sont exprimées sous forme de scénarios exécutables en langage naturel structuré (Gherkin : **Given/When/Then**). Ces scénarios constituent à la fois une spécification et un test d'acceptation, offrant une traçabilité naturelle entre exigences et code. Malgré l'adoption croissante du BDD dans l'industrie, la littérature sur la RTS appliquée aux projets BDD est extrêmement pauvre.

Sur l'ensemble de notre corpus, un seul article répond strictement à la problématique RTS + BDD :

- **A Requirement-based Regression Test Selection Technique in Behavior-Driven Development** [15] (COMPSAC 2021) : approche statique basée sur la traçabilité explicite entre scénarios Gherkin, tests d'acceptation et tests unitaires. Lorsqu'une exigence est modifiée, l'outil propage l'impact pour sélectionner les AT et UT correspondants. La technique est **sûre** et opère à double granularité (scénario et test unitaire).

Deux autres articles intègrent le BDD sans se limiter à la sélection :

- **Test Case Prioritization based on Neural Network Classification with Artifacts Traceability** : exploite la traçabilité BDD pour la *priorisation* (TCP) via un réseau de neurones, mais pas pour la sélection.
- **Automated Regression Tests and Automated Test Optimisation for GetRV** : présente une optimisation globale des tests de régression automatisés à base de scénarios Gherkin.

Cette quasi-absence de travaux confirme que la RTS appliquée au BDD constitue un champ de recherche ouvert, ce qui motive directement les contributions de notre TER.

3.9 Synthèse : matrice de comparaison

Le tableau 1 synthétise les 15 articles retenus selon les six critères issus de nos questions de recherche. Abréviations : **S** = Statique, **D** = Dynamique, **H** = Hybride, **IR** = Recherche d’information; **TC** = Test case, **F** = Fichier, **M** = Méthode, **P** = Package, **Sc** = Scénario+UT; $\checkmark$ = Sûr, $\times$ = Heuristique.

TABLE 1 – Matrice de comparaison des approches RTS

Article	Analyse	Granularité	Sûreté	Algorithmes	Langage	BDD
FeaTestSelPrio	S	Fonctionnalité	$\checkmark$	Graphe + heurist.	C/C++	$\times$
ML Techniques SLR	S+D	TC	$\times$	ML (divers)	Agnostique	$\times$
MultiObj-Mutation	D	TC	$\times$	Évolutionnaire	Java	$\times$
All-Uses BPEL	S	TC	$\checkmark$	Flot de données	BPEL	$\times$
ARTeCS	D	TC	$\times$	Évolutionnaire	Java	$\times$
XCSF (LCS)	D	TC	$\times$	LCS / Renf.	Agnostique	$\times$
ReTEST	IR	TC	$\times$	IR	Agnostique	$\times$
PKRTS	S	Package	$\checkmark$	Graphe	Java	$\times$
SemRTS	S	TC	$\checkmark$	AST + sémantique	Java	$\times$
COMPSAC 2021	S	Sc	$\checkmark$	Traçabilité BDD	Cucumber	$\checkmark$
HyRTS	H	M + F	$\checkmark$	Graphe dynamique	Java	$\times$
AutoTestPlan	IR	TC	$\times$	IR / Lucene	Agnostique	$\times$
ReTSE	S	Instruction	$\checkmark$	Décomp. prog.	C	$\times$
Extraction-Based RTS	S	F	$\checkmark$	Graphe	Java	$\times$
TCP-Traçabilité BDD	D	TC	$\times$	Réseau de neurones	BDD	$\checkmark^*$

* Approche de *priorisation* (TCP), pas de sélection stricto sensu.

La matrice met en évidence plusieurs tendances structurantes :

- **Dominance de l’analyse statique** : 8 articles sur 15 reposent sur une analyse statique, qui garantit systématiquement la propriété de sûreté.
- **Granularité test case majoritaire** : 9 articles sur 15 sélectionnent les tests dans leur ensemble, au détriment d’une précision fine.
- **Faible couverture du BDD** : seuls 2 articles sur 15 ciblent explicitement les projets BDD, dont 1 en priorisation uniquement, confirmant le vide identifié en RQ6.
- **Java dominant** : 6 articles expérimentent sur Java; les approches IR et ML sont quant à elles indépendantes du langage.
- **Compromis sûreté / réduction** : les approches sûres (statiques) tendent à être moins réductives que les approches heuristiques (ML, IR), qui sacrifient la garantie d’exhaustivité pour un taux de réduction plus élevé.

4 Approches Scientifiques

4.1 Approche : RTS pour tests unitaires par analyse statique de dépendances

Cette approche, inspirée de *Extraction-Based RTS* [6], sélectionne sûrement les tests unitaires Java impactés par un commit via l’analyse statique d’un graphe de dépendances entre fichiers, sans instrumentation ni exécution préalable.

Formalisation. Soit F l’ensemble des fichiers source du projet et $T \subseteq F$ les fichiers de test. On construit un graphe orienté $G = (F, D)$ où $(a, b) \in D$ signifie que a dépend de b . Les dépendances sont extraites statiquement depuis trois constructions Java : l’héritage (extends/implements), l’instanciation (new), et l’accès aux membres statiques.

À chaque commit, on calcule M (fichiers modifiés) et A (fichiers ajoutés) depuis le diff Git. L’ensemble des tests sélectionnés est :

$$T_{rts} = T \cap \text{Cl}_{G^{-1}}^*(M \cup A)$$

où G^{-1} est le graphe inversé et Cl^* la fermeture transitive. Le graphe est persisté en JSON et mis à jour de façon **incrémentale** à chaque commit : seuls les arcs des fichiers modifiés sont recalculés, ce qui rend le coût proportionnel à $|M \cup A|$ et indépendant de $|F|$.

Propriété de sûreté. L’approche est **sûre** sous hypothèse de complétude du graphe : tout test impacté par une modification appartient à T_{rts} . Les limites connues sont les dépendances implicites non visibles dans l’AST (réflexion, injection de dépendances), documentées dans la littérature [13].

4.2 Approche : RTS-BDD par similarité sémantique

La question centrale de cet outil est la suivante : *lorsqu’une nouvelle exigence est ajoutée ou modifiée, quels scénarios BDD existants sont suffisamment proches pour devoir être ré-exécutés, et quels tests faut-il relancer ?*

Inspiration : l’approche BDDRTS (COMPSAC 2021). L’approche retenue s’inspire directement de l’article **A Requirement-based Regression Test Selection Technique in Behavior-Driven Development** [15] (Xu, Du et Li). Les auteurs observent que le BDD établit, par construction, une **chaîne de traçabilité** entre les exigences et les tests à deux niveaux de granularité complémentaires.

1. **Niveau acceptation (AT).** Chaque scénario Gherkin est relié à un fichier de *step definitions* via les annotations Cucumber (@Given, @When, @Then). La correspondance est établie en convertissant chaque patron d’annotation en expression régulière et en le comparant aux étapes (*steps*) du scénario considéré.
2. **Niveau unitaire (UT).** Depuis ce fichier de step definitions, les imports Java permettent de remonter jusqu’aux tests unitaires correspondants : chaque classe

importée (hors frameworks Cucumber/JUnit/Spring) est mise en correspondance avec les fichiers `*Test.java` ou `Test*.java` du projet.

Lorsqu’une exigence est modifiée, l’outil propage l’impact le long de cette chaîne : exigence modifiée → step definitions associées → tests unitaires couverts. La technique est **sûre** : tout lien de traçabilité existant garantit la sélection de l’AT et des UT correspondants, sans risque d’omission.

Notre adaptation : similarité sémantique sans traçabilité explicite. L’approche BDDRTS2021 suppose que la modification porte sur un scénario *déjà présent* dans le projet, ce qui permet d’identifier directement le lien de traçabilité. Notre outil traite un cas plus général : la nouvelle exigence n’est pas encore dans le projet, et aucun lien pré-établi n’existe. Plutôt que de s’appuyer sur une traçabilité explicite, notre outil calcule automatiquement, pour chaque nouvelle exigence, les scénarios existants les plus proches **sémantiquement**, puis applique le même mécanisme de propagation AT/UT. L’originalité de l’approche repose sur trois choix :

- **Boîte noire** : aucun accès au code source n’est requis. L’outil raisonne uniquement sur le texte des scénarios Gherkin, le rendant indépendant du langage de programmation.
- **Vectorisation TF-IDF** : chaque scénario est représenté par un vecteur de termes pondérés ($TF \times IDF$), après normalisation NLP (tokenisation, suppression des mots vides, racinisation via l’algorithme de Porter).
- **Seuil adaptatif (knee point)** : le nombre de scénarios retenus est détecté automatiquement comme le point de rupture maximal dans la courbe des scores triés par ordre décroissant, sans paramétrage manuel.

4.3 RTS BDD via LLM

Comme établi dans l’état de l’art, la littérature scientifique ne propose qu’une solution unique à la sélection de tests de non-régression pour le BDD. Cette solution adopte une approche black-box : le code source n’est pas accessible, et la sélection ne peut être déclenchée que par la soumission d’une nouvelle exigence. Cette contrainte structurelle rend une automatisation complète impossible, la majorité des commits dans un cycle de développement réel correspondant à des modifications du code existant plutôt qu’à l’ajout de nouveaux scénarios.

Par ailleurs, les approches de RTS génériques opèrent sur le code source, tandis que les scénarios BDD décrivent des comportements fonctionnels en langage naturel. Ces deux représentations étant totalement opposées, aucun lien direct ne permet d’associer automatiquement une modification de code à un scénario précis, rendant toute sélection fine impossible.

À partir de ce constat, nous proposons une approche white-box originale, pour laquelle il n’existe, à notre connaissance, aucun précédent dans la littérature scientifique. L’accès au code source constitue ici un levier fondamental : il permet d’analyser directement la nature et la portée des modifications introduites par un commit, sans dépendre

d'une action externe telle que la soumission d'une nouvelle exigence. Afin d'établir le lien entre ces modifications structurelles et les comportements fonctionnels décrits en langage naturel, nous tirons parti de la puissance des grands modèles de langage (LLM), dont la capacité de raisonnement sémantique est particulièrement adaptée à la nature hétérogène de ce problème. Cette combinaison ouvre la voie à une automatisation complète du processus de sélection, déclenchée à chaque commit et ne nécessitant aucune intervention humaine..

4.3.1 pourquoi les LLM sont adaptés au problème

Le défi central du RTS dans un contexte BDD est de nature sémantique : il s'agit d'établir un pont entre deux représentations totalement opposées d'un même système d'un côté des modifications exprimées dans un langage de programmation, de l'autre des comportements fonctionnels décrits en langage quasi-naturel (Gherkin). Les approches classiques d'analyse statique ou dynamique opèrent exclusivement dans le domaine structurel du code et ne disposent pas des moyens de traverser cette frontière sémantique.

La puissance des LLMs permet d'opérer à la fois sur du code source et du langage naturel, ils peuvent raisonner sur la signification fonctionnelle d'une modification et l'associer à des comportements décrits. La proximité du langage Gherkin avec le langage naturel renforce cette adéquation.

Cependant, soumettre l'intégralité des scénarios d'un projet à un LLM pour chaque commit présente deux limites : le coût du raisonnement croît avec le nombre de scénarios, et la qualité du raisonnement se dégrade à mesure que le contexte s'allonge. C'est pourquoi nous proposons une approche en deux phases, décrite dans la section suivante.

4.3.2 L'approche hybride : analyse statique et raisonnement LLM

L'approche que nous proposons repose sur une architecture à deux couches successives et complémentaires. Chacune opère sur un registre différent, structurel pour la première, sémantique pour la seconde, et les limites de l'une sont compensées par les forces de l'autre.

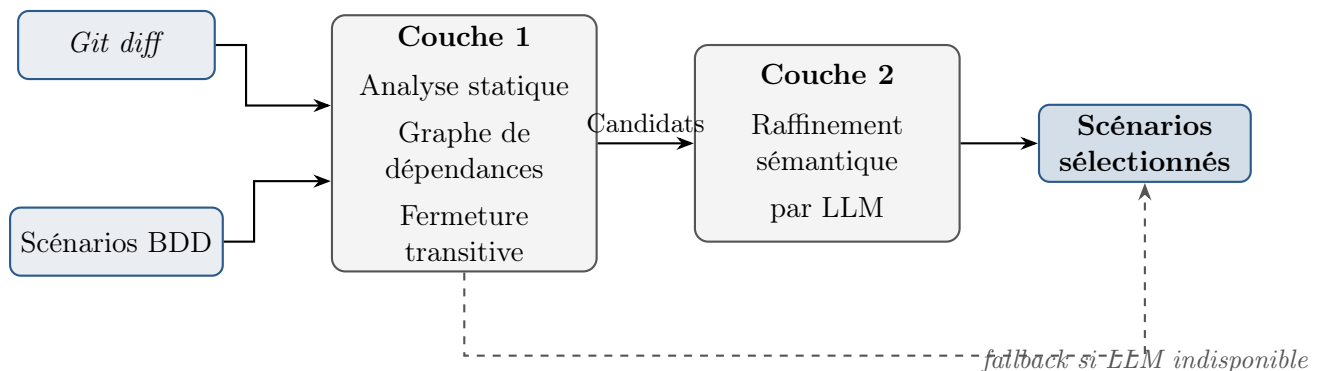


FIGURE 2 – Vue d'ensemble de l'approche hybride de sélection de tests BDD.

Couche 1 : Filtrage structurel par analyse statique. La première couche vise à présélectionner les scénarios qui entretiennent un lien structurel avec les composants modifiés (classes, méthodes...).

Pour cela, un graphe de dépendances est construit à partir du code source, de la même manière que l’approche décrite pour les tests unitaires. Ce graphe représente les relations entre les composants du système : un arc de A vers B signifie que A dépend de B . Lorsqu’un commit est soumis, les composants directement modifiés sont identifiés, puis la **fermeture transitive** de leurs dépendants est calculée par parcours du graphe inversé. Cet ensemble représente tous les composants susceptibles d’être affectés, directement ou indirectement.

Les scénarios dont les définitions d’étapes impliquent au moins un composant de cet ensemble sont retenus comme candidats. Cette couche présente une propriété essentielle : elle est sûre par construction, c’est à dire que l’espace des candidats possède un rappel de 100%. Aucun scénario structurellement lié à une modification ne peut être exclu à ce stade. En contrepartie, elle peut retenir des scénarios qui, bien que connectés structurellement, ne sont pas fonctionnellement affectés c’est le rôle de la seconde couche de les écarter.

Couche 2 : Raffinement sémantique par raisonnement LLM. Le rôle de cette couche va être de maximiser la précision, c’est à dire élaguer les faux positifs (scénarios non impactés par les modification) de l’espace des candidats. Cette seconde couche prend en entrée le sous-ensemble de candidats produit par la couche statique, ainsi que la description des modifications introduites par le commit. Elle soumet ces éléments à un LLM pour un raffinement sémantique.

La question posée au modèle est formulée de manière **conservatrice** : il ne lui est pas demandé quels scénarios sont impactés, mais quels scénarios peuvent être ignorés en toute sécurité, avec l’obligation de justifier chaque exclusion. Cette inversion limite les faux négatifs : le modèle ne peut écarter un scénario qu’en produisant une justification explicite, ce qui réduit les exclusions erronées dues à un raisonnement superficiel.

En cas d’indisponibilité du LLM, la couche statique seule constitue un repli valide, préservant la sûreté de la sélection.

Propriétés de l’approche hybride. La combinaison des deux couches tire parti de leurs complémentarités :

- L’analyse statique garantit l’exhaustivité structurelle et produit un résultat déterministe, sans coût d’inférence.
- Le raisonnement LLM apporte une capacité d’élimination sémantique que l’analyse structurelle seule ne peut offrir, réduisant les faux positifs résiduels.
- La réduction préalable de l’espace de candidats par la couche statique permet de limiter le contexte soumis au LLM, améliorant la qualité du raisonnement et réduisant son coût.

5 Implémentation

5.1 RTS sur les tests unitaires

Le premier outil implémente l'approche décrite en Section 4.1. Il prend en entrée un projet Java et le hash d'un commit, et produit la liste des fichiers de test à ré-exécuter ainsi qu'une commande Maven prête à l'emploi. Il est entièrement non intrusif : il lit le code source et l'historique Git en lecture seule, sans modifier le `pom.xml` ni nécessiter d'instrumentation préalable.

5.1.1 Intérêt de l'outil

La sélection de tests unitaires par analyse statique est déjà documentée, mais les outils de référence (*Ekstazi*, *STARTS*) s'intègrent via un plugin Maven qui modifie en profondeur le build du projet cible. Cette intrusion est un obstacle en contexte d'expérimentation, où l'on veut évaluer rapidement plusieurs projets open source sans patcher leur configuration. Notre outil adopte l'approche inverse : un utilitaire externe qui délègue l'exécution des tests à la commande Maven standard.

Cet outil constitue par ailleurs la première brique d'une progression logique : il introduit les composants techniques, analyse AST (JavaParser), extraction de diff (JGit), parcours de graphe de dépendances, réutilisés et étendus dans l'outil BDD via LLM .

5.1.2 Fonctionnement de l'outil

Le pipeline s'organise en quatre phases, illustrées par la figure 3.

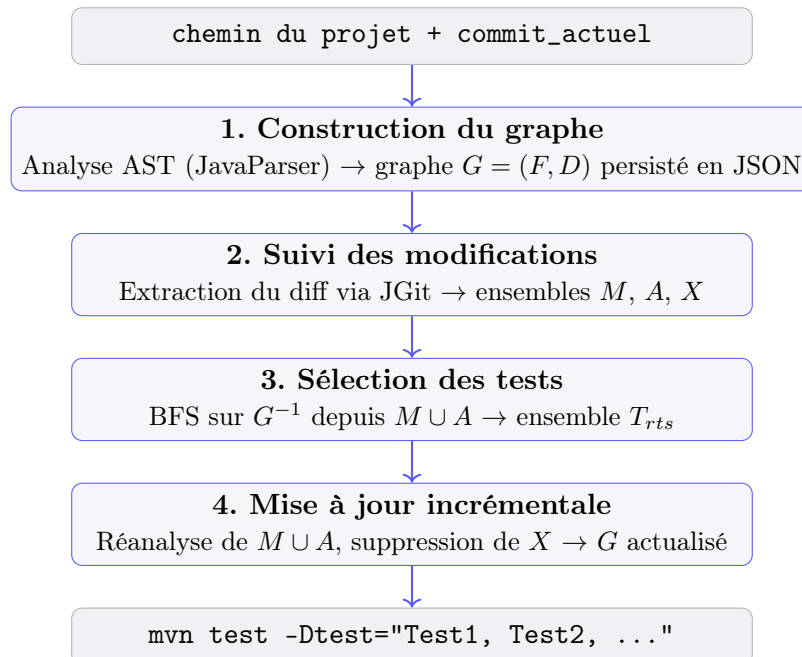


FIGURE 3 – Pipeline de traitement de l'outil RTS pour tests unitaires.

1. Construction du graphe. Au premier passage, l'outil parcourt `src/` et extrait l'AST de chaque fichier `.java` via `JavaParser`. Pour chaque fichier, trois types de dépendances sont identifiés : les imports internes au projet, les relations d'héritage (`extends/implements`) et les instanciations explicites. Le graphe $G = (F, D)$ obtenu est sérialisé en JSON pour être rechargé lors des invocations suivantes, sans ré-analyse complète.

2. Suivi des modifications. L'outil compare, via `JGit`, l'arbre du commit courant à celui de son parent, et partitionne les fichiers Java en trois ensembles : modifiés (M), ajoutés (A) et supprimés (X). Le recours à l'API `JGit` plutôt qu'à un appel `git diff` externe évite toute dépendance à un binaire et donne un accès fin aux métadonnées du diff.

3. Sélection des tests. C'est le cœur de l'outil. Depuis $M \cup A$, un parcours en largeur (BFS) sur le graphe *inversé* G^{-1} calcule la fermeture transitive des dépendants des fichiers modifiés. Cet ensemble est filtré pour ne conserver que les fichiers de test (convention `JUnit : *Test.java` ou `Test*.java`). La complexité linéaire en $|F| + |D|$ rend l'analyse rapide même sur de gros projets.

4. Mise à jour incrémentale. Pour préparer le commit suivant, l'outil retire du graphe les fichiers supprimés, recalcule les dépendances des fichiers modifiés ou ajoutés, et re-séréalise le graphe. Le surcoût entre deux commits reste ainsi proportionnel au volume du commit, et non à la taille du projet.

5.1.3 Architecture logicielle

L'outil suit une séparation des préoccupations en deux couches. La **couche service** orchestre le traitement (`GraphBuilder`, `DiffExtractor`, `TestSelector`, `GraphUpdater`) ; la **couche modèle** porte les données (`DependencyGraph`, `Commit`, `RTSResult`). Le diagramme de classes (figure 4) illustre cette organisation.

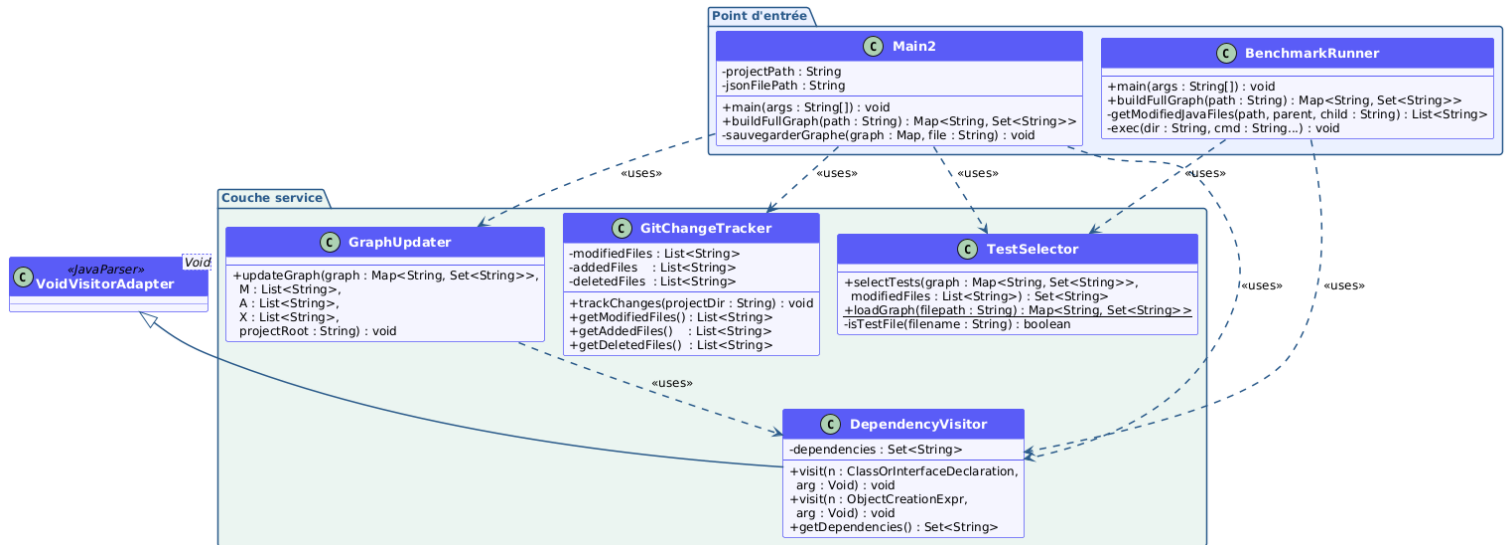


FIGURE 4 – Diagramme de classes de l'outil RTS pour tests unitaires.

5.1.4 Résultat et utilisation

L'outil s'invoque via des propriétés système Maven, sans modifier le projet cible, et produit un rapport listant les tests à ré-exécuter avec leur justification.

```
1 =====
2 SÉLECTION RTS pour le commit a3f2c91
3 =====
4 Fichiers modifiés : src/main/java/com/example/Calculator.java
5
6 Tests unitaires à ré-exécuter : 3
7   - CalculatorTest          (dépend de Calculator)
8   - CalculatorIntegrationTest (via CalculatorService)
9   - ScientificCalculatorTest (Calculator est superclasse)
10
11 Tests non sélectionnés : 12 fichiers (80% économisés)
12
13 → mvn test -Dtest="CalculatorTest,CalculatorIntegrationTest,
14               ScientificCalculatorTest"
15 =====
```

Listing 1 – Exemple de rapport produit par l'outil.

5.1.5 Limitations et transition vers les approches BDD

L'outil présente deux limitations qui motivent directement les approches suivantes.

Granularité fichier. L'outil ne sélectionne que des tests unitaires. Sur un projet BDD, le runner Cucumber apparaît comme une unique classe `RunCucumberTest`, sélectionnée ou non en bloc, exactement comme `Ekstazi` ou `STARTS`. Aucune sélection au niveau du scénario n'est possible, car le graphe ne descend pas sous le fichier Java. C'est ce que lèvera l'outil BDD boîte noire.

Dépendances implicites. L'analyse statique ne voit pas les dépendances établies par réflexion ou injection (`@Autowired` de Spring, `Class.forName...`). Sur les projets Spring, ces mécanismes produisent des faux négatifs : des tests impactés ne sont pas sélectionnés. Cette limite, commune à toutes les approches statiques, justifie le recours à un LLM dans l'approche hybride, capable de raisonner partiellement sur ces dépendances implicites.

5.2 BDD black box

Notre premier outil BDD implémente l'approche décrite ci-dessus. Il prend en entrée un fichier `.feature` décrivant la nouvelle exigence et un projet BDD existant, et produit en sortie la liste des tests d'acceptation et unitaires à ré-exécuter, accompagnée d'une commande Maven prête à l'emploi.

5.2.1 Intérêt de l'outil

Actuellement, aucun outil grand public ne permet de sélectionner des scénarios BDD individuels à partir d'une nouvelle exigence, sans accès au code source. Les approches existantes de RTS (Ekstazi, STARTS) opèrent au niveau des classes Java et traitent le runner Cucumber comme un bloc monolithique : elles sont incapables de distinguer quel scénario individuel est sémantiquement lié à une nouvelle exigence.

Notre outil comble ce manque en permettant une sélection fine au niveau du scénario, sans nécessiter d'instrumentation du code ni de modification du projet cible. Il suffit de fournir le répertoire des `.feature` existants, le répertoire des sources de test Java et le fichier décrivant la nouvelle exigence. L'outil fonctionne sur n'importe quel projet BDD Java et peut s'intégrer dans une pipeline CI/CD.

5.2.2 Fonctionnement de l'outil

L'outil analyse la nouvelle exigence et détermine quels scénarios BDD existants sont sémantiquement proches et doivent être ré-exécutés. Le processus se déroule en six étapes, illustrées par le pipeline de la figure 5.

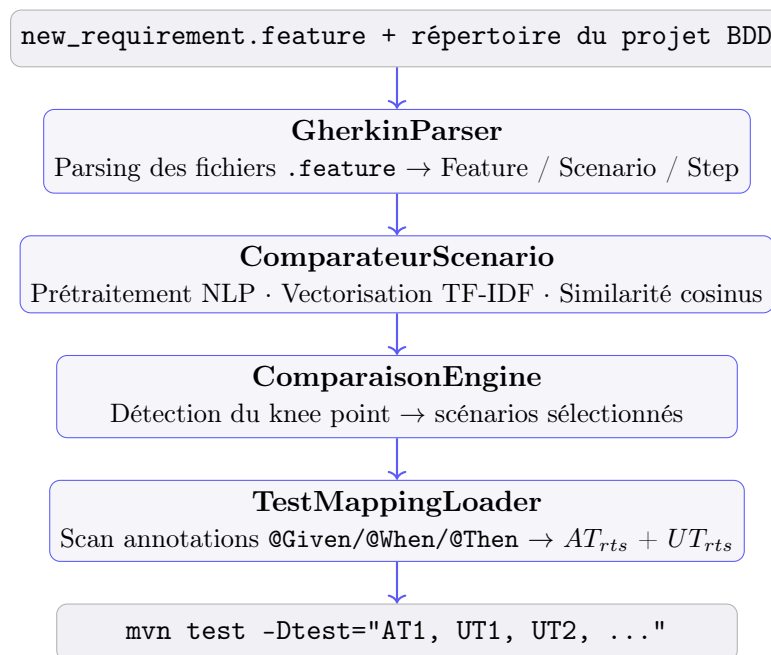


FIGURE 5 – Pipeline de traitement de l'outil RTS-BDD black box

Le diagramme de classes (figure 6) illustre l'architecture de l'outil, organisée en deux couches : une **couche service** (GherkinParser, ComparateurScenario, ComparaisonEngine, TestMappingLoader) qui orchestre le traitement, et une **couche modèle** (Feature, Scenario, Step, RTSResult) qui porte les données parsées et le résultat final.

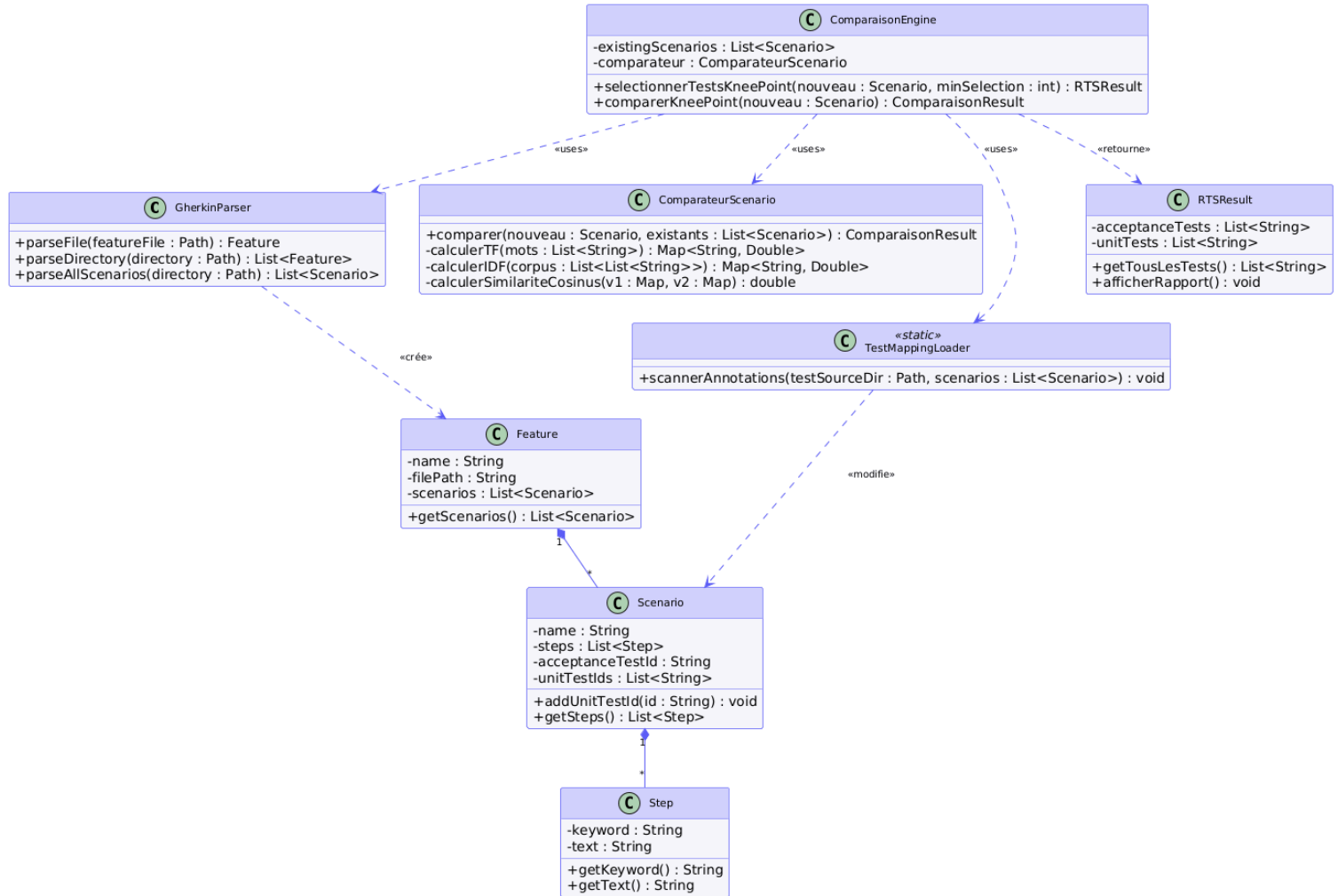


FIGURE 6 – Diagramme de classes de l'outil RTS-BDD black box

Le diagramme de séquence (figure 7) décrit les échanges entre composants en deux phases : l'**initialisation** (parsing du corpus de scénarios et injection des identifiants AT/UT par TestMappingLoader), puis l'**analyse** de la nouvelle exigence (comparaison TF-IDF déléguée à CompareurScenario, détection du knee point, retour du RTSResult).

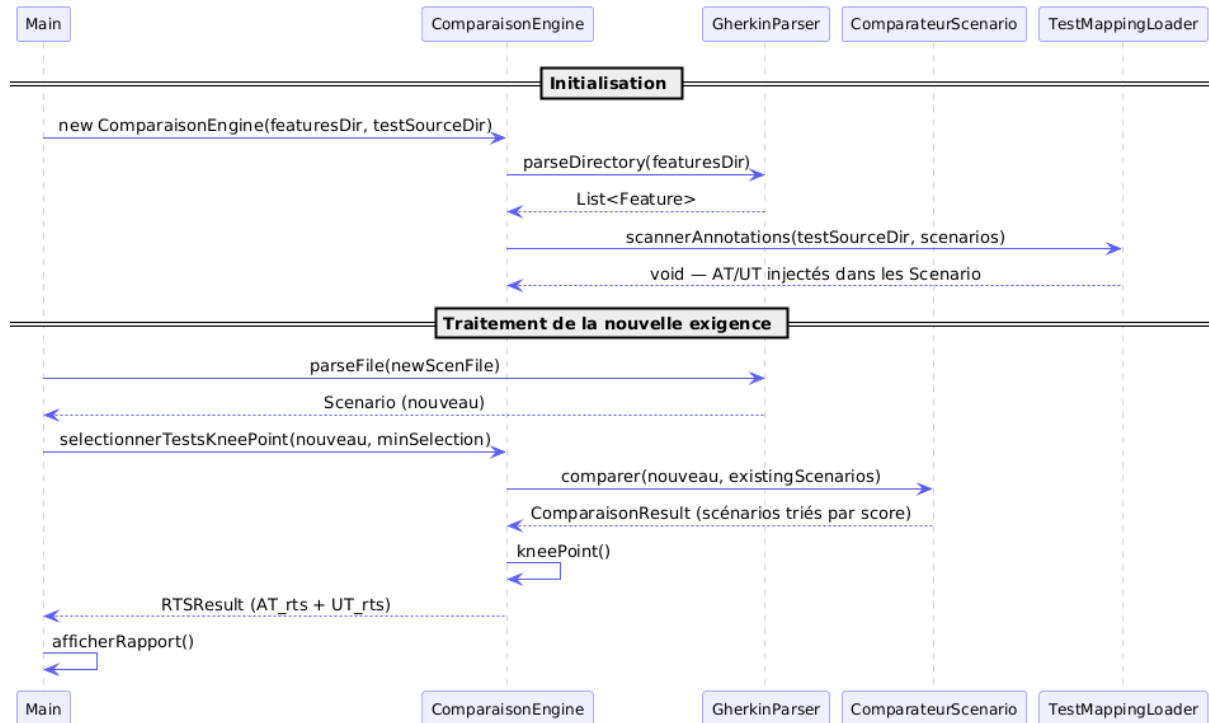


FIGURE 7 – Diagramme de séquence de l'outil RTS-BDD black box

5.2.3 Parsing des scénarios existants

Le composant `GherkinParser` parcourt récursivement le répertoire des fichiers `.feature` du projet cible. Il construit un corpus de scénarios : chaque scénario est représenté par son nom et la liste ordonnée de ses steps (mot-clé + texte brut). Le parseur supporte les mots-clés `Given`, `When`, `Then`, `And`, `But` ainsi que les `Scenario Outline`. Le fichier de la nouvelle exigence est parsé de la même façon, ce qui garantit un traitement homogène.

Voici un exemple de fichier `new_requirement.feature` passé en entrée de l'outil :

```

1 Feature: New arithmetic operation
2
3   Scenario: Computing the modulo of two integers
4     Given an integer operation '%'
5     When I provide a first number 7
6     And I provide a second number 3
7     Then the operation evaluates to 1
  
```

Listing 2 – Exemple de nouvelle exigence (`new_requirement.feature`)

5.2.4 Prétraitement du texte

Le texte de chaque scénario (nom concaténé au texte de l'ensemble de ses steps) est prétraité à l'aide de l'EnglishAnalyzer d'Apache Lucene, qui réalise automatiquement trois opérations :

- **Tokenisation** : découpage du texte en unités lexicales (tokens) ;
- **Suppression des mots vides** (*stop words*) : élimination des mots non discriminants (articles, prépositions, etc.) ;
- **Racinisation** (*stemming*) : réduction de chaque mot à sa racine selon l'algorithme de Porter (ex. : « running » → « run »).

Cette normalisation garantit que deux scénarios exprimant la même idée avec des formulations légèrement différentes obtiennent un score de similarité élevé.

5.2.5 Vectorisation TF-IDF

Chaque scénario prétraité est représenté par un vecteur TF-IDF (*Term Frequency – Inverse Document Frequency*). Pour chaque terme t du vocabulaire :

- $TF(t, s)$: fréquence du terme t dans le scénario s , normalisant la longueur du scénario ;
- $IDF(t) : \log\left(\frac{N}{df(t)}\right)$ où N est le nombre de scénarios et $df(t)$ le nombre de scénarios contenant t .

Le poids final de chaque terme est $TF(t, s) \times IDF(t)$, ce qui donne plus de poids aux termes spécifiques à un scénario et peu communs dans le corpus.

L'implémentation Java de ces deux fonctions est la suivante :

```
1 private Map<String, Double> calculerTF(List<String> mots) {
2     Map<String, Double> result = new HashMap<>();
3     int nbMot = mots.size();
4     for (String s : mots)
5         result.put(s, result.getOrDefault(s, 0.0) + 1);
6     for (Map.Entry<String, Double> e : result.entrySet())
7         e.setValue(e.getValue() / nbMot);
8     return result; }
9 private Map<String, Double> calculerIDF(List<List<String>>
10    tousLesMots) {
11     Map<String, Double> result = new HashMap<>();
12     int N = tousLesMots.size();
13     for (List<String> scenario : tousLesMots) {
14         Set<String> uniques = new HashSet<>(scenario);
15         for (String s : uniques)
16             result.put(s, result.getOrDefault(s, 0.0) + 1); }
17     for (Map.Entry<String, Double> e : result.entrySet())
18         e.setValue(Math.log((double) N / e.getValue()));
19     return result; }
```

Listing 3 – Calcul du TF et de l'IDF (CompareurScenario.java)

5.2.6 Calcul de la similarité cosinus

La proximité sémantique entre la nouvelle exigence s_{new} et chaque scénario existant s_i est calculée par la **similarité cosinus** des vecteurs TF-IDF :

$$sim(s_{new}, s_i) = \frac{\vec{v}_{new} \cdot \vec{v}_i}{\|\vec{v}_{new}\| \cdot \|\vec{v}_i\|}$$

Un score de 1,0 indique une identité sémantique parfaite; 0,0 indique l'absence totale de termes communs. Les scénarios existants sont ensuite triés par score décroissant pour former un classement de similarité.

```
1 private double calculerSimilariteCosinus(Map<String, Double> v1,
2                                         Map<String, Double> v2) {
3     double dot = 0.0, n1 = 0.0, n2 = 0.0;
4     for (Map.Entry<String, Double> e : v1.entrySet()) {
5         n1 += e.getValue() * e.getValue();
6         if (v2.containsKey(e.getKey()))
7             dot += e.getValue() * v2.get(e.getKey());
8     }
9     for (double v : v2.values()) n2 += v * v;
10    if (n1 == 0.0 || n2 == 0.0) return 0.0;
11    return dot / (Math.sqrt(n1) * Math.sqrt(n2));
12 }
```

Listing 4 – Similarité cosinus entre deux vecteurs TF-IDF (CompareurScenario.java)

5.2.7 Détection du knee point

Plutôt que de fixer arbitrairement un seuil ou un nombre de scénarios à sélectionner, l'outil détecte automatiquement le *point de coude* (*knee point*) dans la courbe des scores triés. Ce point correspond au plus grand écart entre deux scores consécutifs : au-delà, les scénarios sont jugés trop peu similaires pour être pertinents. Un paramètre `minSelection` (par défaut 3) garantit qu'un nombre minimal de scénarios est toujours retourné, même lorsque les scores sont uniformément proches.

```
1 int kneeIndex = ranking.size();
2 double maxDrop = 0;
3 for (int i = 1; i < ranking.size(); i++) {
4     double drop = ranking.get(i - 1).getScoreSimilarite()
5                 - ranking.get(i).getScoreSimilarite();
6     if (drop > maxDrop) {
7         maxDrop = drop;
8         kneeIndex = i;
9     }
10 }
11 int cutoff = Math.max(kneeIndex, minSelection);
12 cutoff = Math.min(cutoff, ranking.size());
13 return ranking.subList(0, cutoff);
```

Listing 5 – Détection du knee point (ComparaisonEngine.java)

5.2.8 Mapping AT/UT

Une fois les scénarios similaires sélectionnés, l'outil remonte la chaîne de traçabilité AT/UT décrite en section Approche. L'implémentation repose sur deux expressions régulières :

```
1 private static final Pattern STEP_ANNOTATION = Pattern.compile(  
2     "@(Given|When|Then|And|But)\\s*\\(\"(.+?)\\\"\\)");  
3  
4 private static final Pattern IMPORT_PATTERN = Pattern.compile(  
5     "import\\s+([\\w.]+)\\s*;" );
```

Listing 6 – Patterns de détection des annotations et des imports (TestMappingLoader.java)

- **Tests d'acceptation (AT)** : TestMappingLoader scanne les fichiers `*Steps.java` du projet. Pour chaque annotation trouvée via `STEP_ANNOTATION`, le pattern Cucumber est converti en regex (ex. : `{int} → \\d+`) et comparé aux steps des scénarios sélectionnés. Le fichier steps qui matche est retenu comme AT à relancer.
- **Tests unitaires (UT)** : depuis le fichier steps identifié, `IMPORT_PATTERN` extrait les classes importées (hors frameworks Cucumber, JUnit, Spring). Chaque classe est mise en correspondance avec les fichiers `XxxTest.java` (convention JUnit 5) ou `TestXxx.java` (convention JUnit 4), permettant de remonter jusqu'aux tests unitaires sous-jacents.

5.2.9 Résultat et utilisation

Pour chaque nouvelle exigence, l'outil affiche un rapport structuré listant les scénarios similaires retenus avec leur score, les tests d'acceptation (AT_{rts}) et unitaires (UT_{rts}) à ré-exécuter, et génère automatiquement la commande Maven correspondante.

L'outil est entièrement générique et s'invoque via des propriétés système Maven, sans modifier le projet cible :

```
1 mvn compile exec:java \  
2   -Dexec.mainClass="com.rts.MainWithAnnotations" \  
3   -Drts.features=<chemin/vers/features> \  
4   -Drts.testSource=<chemin/vers/src/test/java> \  
5   -Drts.newScenario=<chemin/vers/new_requirement.feature> \  
6   -Drts.minSelection=3
```

Listing 7 – Commande d'exécution de l'outil RTS-BDD black box

Voici un exemple de rapport produit par l'outil sur le projet `calculator-cucumber` :

SÉLECTION RTS pour : Computing the modulo of two integers

Scénarios similaires sélectionnés : 3

- [0.4821] Provide two numbers and an operation ("+")
- [0.3714] Provide two numbers and an operation ("-")
- [0.3102] Provide two numbers and an operation ("*")

Tests d'acceptation à ré-exécuter (AT_rts) : 1

- CalculatorSteps

Tests unitaires à ré-exécuter (UT_rts) : 3

- TestPlus
- TestMinus
- TestMultiplies

→ Commande : `mvn test -Dtest="CalculatorSteps,TestPlus,TestMinus,TestMultiplies"`

5.2.10 Limitations

L'approche black box présente deux limitations inhérentes liées à ses choix de conception.

Mapping AT/UT conditionnel. Le mécanisme de remontée vers les tests repose sur la présence de fichiers `*Steps.java` annotés avec les annotations Cucumber (`@Given`, `@When`, `@Then`). Pour des frameworks comme **Karate DSL**, qui utilisent une syntaxe déclarative sans annotations Java, le mapping reste vide : l'outil sélectionne des scénarios similaires mais ne peut pas identifier les tests correspondants. Seule la similarité TF-IDF guide alors la sélection.

Limite de la comparaison textuelle. L'approche TF-IDF mesure une proximité *lexicale*, pas une proximité *sémantique profonde*. Deux scénarios fonctionnellement liés mais rédigés avec un vocabulaire différent (sans termes communs après stemming) obtiendront un score nul malgré leur pertinence. À l'inverse, deux scénarios partageant des mots communs génériques peuvent obtenir un score élevé sans lien fonctionnel réel. C'est précisément pour surmonter cette limite que l'outil BDD LLM a été développé en complément.

5.3 BDD LLM

Après avoir implémenté deux outils, respectivement pour les tests unitaires et les tests BDD, nous avons décidé de créer un outil se basant sur une toute nouvelle approche, non documenté.

5.3.1 Intérêt de l'outil

Actuellement, il n'existe pas d'outil grand public permettant la sélection de tests BDD basé sur les modifications de code source. Les outils de RTS existants comme Ekstazi ou STARTS fonctionnent au niveau des classes Java : ils traitent le runner Cucumber comme un bloc monolithique et sont incapables de distinguer quel scénario individuel est impacté par un changement de code. Le choix se limite alors à rejouer tous les scénarios ou aucun.

La puissance des LLMs vont nous permettre d'analyser directement les modifications de code entre deux commits, et raisonner sur son impact fonctionnel pour identifier les scénarios concernés. Contrairement à l'approche TF-IDF de l'outil précédent qui compare des textes entre eux, le LLM comprend la logique métier ainsi que l'impact de modifications sur le code source.

Cet outil permet de répondre à cette problématique, capable de s'intégrer dans une pipeline CI/CD et de fonctionner sur n'importe quel commit. L'implémentation de l'approche s'est déroulée en deux étapes itératives. Nous avons d'abord développé une version reposant uniquement sur les LLM, afin de valider le principe de base. Nous avons ensuite enrichi cette première version d'une phase d'analyse statique en amont, pour pallier aux limites observées.

5.3.2 Première version : sélection par le LLM seul

La première implémentation suit une logique directe : à partir d'un commit, extraire le diff Git et l'ensemble des scénarios BDD du projet, puis soumettre ces deux éléments à un LLM en lui demandant d'identifier les scénarios susceptibles d'être impactés.

Fonctionnement général

Pour chaque commit analysé, le pipeline se déroule en trois étapes :

1. **Extraction du diff.** La commande `git diff` est exécutée entre le commit parent et le commit courant. Le diff obtenu contient les lignes ajoutées et supprimées dans chaque fichier modifié.
2. **Collecte des scénarios BDD.** L'outil parcourt les fichiers `.feature` du projet et en extrait l'ensemble des scénarios avec leurs steps Gherkin.
3. **Appel au LLM.** Un prompt est construit en concaténant le diff et la liste des scénarios numérotés. Le modèle est invité à retourner les indices des scénarios dont le comportement pourrait être affecté par les modifications, accompagnés d'une justification.

Construction du prompt

Le prompt soumis au LLM positionne le modèle comme expert en tests de régression BDD, et lui présente deux blocs d'information : le diff du commit et la liste complète des scénarios. Il lui est demandé de répondre au format JSON structuré :

```
{ "selected": [2, 5], "reasoning": "..." }
```

Ce format contraint la réponse et facilite son exploitation automatique. La réponse est ensuite parsée pour en extraire les indices sélectionnés, qui sont validés contre la liste des scénarios fournis afin d'éviter toute hallucination d'indice. Le champ reasoning permet de renforcer la cohérence de la sélection en obligeant le modèle à formuler explicitement sa justification. Un LLM qui doit argumenter chaque choix est moins susceptible de sélectionner un scénario sans lien réel avec les modifications.

Modèle utilisé et configuration

L'outil a été conçu pour fonctionner avec plusieurs fournisseurs de LLM (OpenAI, Anthropic, Gemini, Ollama). Pour l'évaluation, nous avons utilisé **Mistral 7B** via Ollama, qui permet une exécution locale sans dépendance à une API payante. La température est fixée à 0,1 pour garantir un comportement quasi-déterministe et améliorer la reproductibilité des résultats.

Limites observées

Cette première approche présente deux limites importantes, mises en évidence lors de l'évaluation :

- **Passage à l'échelle difficile.** Le prompt contient l'intégralité des scénarios du projet. À mesure que ce nombre croît, le contexte soumis au modèle s'allonge, ce qui dégrade la qualité du raisonnement et augmente les coûts d'inférence.
- **Manque de précision.** Sans connaissance de la structure du code, le LLM ne peut relier un changement dans une méthode à un scénario qu'en s'appuyant sur des indices lexicaux dans le diff. Il tend à sur-sélectionner (faux positifs) par prudence, ou à manquer des impacts indirects non apparents dans le diff (faux négatifs).

Ces limites ont motivé le développement d'une seconde version intégrant une phase d'analyse statique en amont.

5.3.3 Seconde version : analyse statique en amont du LLM

La seconde version introduit une couche d'analyse statique du code source avant l'appel au LLM. L'idée centrale est de ne soumettre au modèle que les scénarios déjà présélectionnés sur la base d'un lien structurel démontrable avec le code modifié, réduisant ainsi à la fois le bruit et la taille du contexte.

Analyse statique du projet

Avant tout appel au LLM, l'outil analyse le code source du projet cible à l'aide de la bibliothèque **JavaParser**. Cette analyse produit deux structures :

- Un graphe d'appels reliant les méthodes entre elles : un arc de A vers B signifie que la méthode A appelle la méthode B . Ce graphe est également construit en sens inverse (reverse call graph) pour permettre de remonter de n'importe quelle méthode vers ses appelants.
- Une table de correspondance entre les scénarios BDD et les méthodes Java de leurs step definitions : pour chaque scénario, on connaît l'ensemble des méthodes Java invoquées par ses étapes Gherkin.

Identification des scénarios candidats

Lorsqu'un commit est soumis, les méthodes modifiées par le diff sont d'abord identifiées par croisement des lignes modifiées avec les plages de méthodes issues de l'analyse statique. On obtient ainsi un ensemble M de méthodes touchées par le commit.

À partir de M , on remonte le reverse call graph par un parcours en largeur (BFS) pour identifier tous les composants transitivement dépendants des méthodes modifiées. Un scénario est retenu comme candidat si au moins une de ses step definitions appartient à cet ensemble. On obtient ainsi un sous-ensemble $C \subseteq \mathcal{S}$ des scénarios structurellement liés aux modifications.

Par construction, tout scénario qui invoque directement ou indirectement une méthode modifiée est inclus dans C . La couche statique est donc sûre : elle ne peut pas exclure un scénario véritablement impacté (rappel de 100%), sous réserve que le graphe d'appels soit suffisamment complet.

Raffinement par le LLM

Le LLM reçoit ensuite uniquement les scénarios de C , et non plus l'ensemble du projet. Pour chaque candidat, le prompt inclut également la **chaîne d'appel** qui relie sa step definition à la méthode modifiée, par exemple :

```
setBodyTo → AbstractBddStepDefinition.addQueryParameters
```

Cette information permet au LLM de comprendre *comment* le scénario atteint le code modifié, et non seulement *que* ce lien existe. Il peut ainsi distinguer un impact réel d'un faux positif structurel (lien existant dans le graphe, mais modification sans effet sur le comportement observable du scénario, comme un changement de log ou de commentaire).

Apport de la seconde version

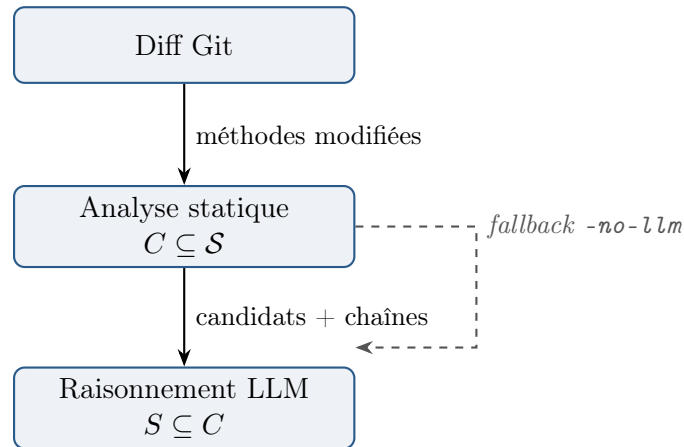


FIGURE 8 – Pipeline de la seconde version : filtrage statique puis raffinement LLM.

La combinaison des deux couches apporte trois améliorations concrètes par rapport à la version LLM seul :

- Le contexte soumis au LLM est réduit aux seuls scénarios pertinents, ce qui améliore la qualité du raisonnement et réduit la latence.
- Le LLM dispose d'une information structurelle précise (chaînes d'appel) qu'il ne pouvait pas déduire seul à partir du diff.
- En cas d'indisponibilité du LLM, l'analyse statique seule (option `-no-llm`) constitue un repli valide qui préserve la sûreté de la sélection.

6 Méthodologie de Validation et Vérité terrain

6.1 Le problème de l'évaluation

Évaluer la qualité d'un outil de sélection de tests de régression pose un problème fondamental : l'outil produit une liste de scénarios à ré-exécuter, mais rien ne permet a priori de savoir si cette liste est bonne. Pour cela, il faudrait un oracle nous permettant de connaître, pour chaque commit, l'ensemble des scénarios qui auraient réellement dû être ré-exécutés.

Cet oracle est appelé vérité terrain. Sans lui, il est impossible de distinguer un scénario correctement sélectionné d'un scénario sélectionné par erreur, ni de détecter les scénarios impactés que l'outil aurait manqués. La construction de cette vérité terrain est donc une étape préalable indispensable à toute évaluation rigoureuse.

6.2 Définition de la vérité terrain

6.2.1 Principe

La vérité terrain repose sur l'observation suivante : un scénario est impacté par un commit si et seulement si son exécution traverse au moins une des méthodes modifiées par ce commit.

Ce principe s'appuie donc sur l'exécution réelle des tests, et non sur une heuristique ou une approximation.

Formellement, pour un diff donné, la vérité terrain G est définie par :

$$G(diff) = \{ s \in \mathcal{S} \mid Couverture(s) \cap MthodesModifies(diff) \neq \emptyset \}$$

où $\mathcal{S}$ est l'ensemble des scénarios du projet, $Couverture(s)$ l'ensemble des méthodes effectivement exécutées lors du passage du scénario s , et $MthodesModifies(diff)$ l'ensemble des méthodes dont le corps est modifié par le diff.

6.2.2 Couverture dynamique avec JaCoCo

Pour mesurer $Couverture(s)$, nous utilisons **JaCoCo**, un outil d'instrumentation du bytecode Java qui enregistre, lors de l'exécution d'un test, l'ensemble exact des méthodes traversées.

Afin de pouvoir utiliser efficacement cette couverture, il est important d'isoler l'exécution des scénarios. Chaque scénario est exécuté dans une JVM distincte, indépendamment des autres. Sans cette isolation, la couverture mesurée pour un scénario pourrait être contaminée par l'exécution d'un scénario précédent (caches en mémoire, état de la base de données...). L'isolation garantit que la couverture mesurée correspond bien au seul scénario considéré.

6.3 Métriques d'évaluation

Pour évaluer la qualité de la sélection produite par RTS-LLM, on compare l'ensemble S des scénarios sélectionnés par l'outil à l'ensemble G de la vérité terrain. Trois situations peuvent se présenter pour chaque scénario :

Sigle	Nom	Signification	Définition
VP	Vrai Positif	Le scénario est sélectionné et réellement impacté. C'est le cas idéal : l'outil a pris la bonne décision.	$ S \cap G $
FP	Faux Positif	Le scénario est sélectionné mais pas réellement impacté. Le test sera ré-exécuté inutilement, ce qui rallonge le temps, sans compromettre la qualité.	$ S \setminus G $
FN	Faux Négatif	Le scénario est impacté mais l'outil ne l'a pas sélectionné. C'est le cas le plus grave : un défaut introduit par le commit pourrait ne pas être détecté et atteindre la production.	$ G \setminus S $

TABLE 2 – Définition des cas de figure de la sélection.

À partir de ces trois quantités, on calcule trois métriques classiques de la recherche d'information :

$$Precision = \frac{VP}{VP + FP} \quad Rappel = \frac{VP}{VP + FN} \quad F_1 = \frac{2 \times Precision \times Rappel}{Precision + Rappel}$$

La précision mesure la proportion de scénarios sélectionnés qui sont effectivement impactés. Une précision élevée signifie que l'outil sélectionne peu de scénarios inutiles : le pipeline de CI reste léger. Une précision faible indique que l'outil est trop conservateur et sélectionne beaucoup de scénarios sans lien réel avec les modifications.

Le rappel mesure la proportion de scénarios impactés que l'outil a effectivement sélectionnés. Un rappel élevé signifie que peu de scénarios impactés ont été oubliés. C'est l'indicateur prioritaire pour un outil RTS : un faux négatif (rappel insuffisant) signifie qu'un scénario impacté n'est pas ré-exécuté, permettant potentiellement à un défaut d'atteindre la production sans être détecté. Un faux positif (précision insuffisante), en revanche, n'entraîne qu'une exécution superflue dont le coût est borné et prévisible.

Le score F1 est la moyenne harmonique de la précision et du rappel. Il offre une vision synthétique de la qualité globale de la sélection, pénalisant les outils qui optimiseraient l'une des deux métriques au détriment de l'autre (par exemple, un outil qui sélectionne systématiquement tous les scénarios aurait un rappel parfait de 1 mais une précision très faible).

Dans le contexte du RTS, un rappel de 1 est toujours atteignable trivialement en sélectionnant l'intégralité des scénarios à chaque commit, mais cela annule tout l'intérêt

de la sélection. L'enjeu est d'obtenir un rappel élevé *tout en maintenant une précision raisonnable*, ce que le score F1 permet de mesurer conjointement.

6.4 Mise en place concrète

6.4.1 Collecte de la couverture par scénario

La première étape consiste à mesurer, une fois pour toutes, la couverture de chaque scénario dans la version de référence du projet. Pour cela, un script exécute chaque scénario Cucumber individuellement, avec l'agent JaCoCo attaché à la JVM. À l'issue de chaque exécution, le fichier de couverture produit est archivé sous un nom unique lié au scénario.

On obtient ainsi une **map de couverture** : pour chaque scénario, la liste des méthodes Java effectivement traversées lors de son passage.

6.4.2 Calcul de la vérité terrain pour un commit

Pour un commit donné, le calcul de G suit trois étapes :

1. **Extraction des méthodes modifiées.** Le diff entre le commit et son parent est analysé pour identifier les méthodes Java dont le corps a été modifié (par croisement des hunks du diff avec les plages de méthodes issues de l'analyse statique).
2. **Intersection avec la map de couverture.** Pour chaque scénario, on vérifie si l'ensemble de ses méthodes couvertes intersecte l'ensemble des méthodes modifiées. Si oui, le scénario appartient à G .
3. **Calcul des métriques.** La sélection S produite par l'outil est comparée à G pour calculer VP, FP, FN, précision, rappel et F1.

6.4.3 Application sur l'historique Git

Pour obtenir un jeu d'évaluation suffisamment représentatif, ce calcul est automatisé sur une plage de commits de l'historique Git du projet cible. Pour chaque commit, la couverture est collectée dans l'état du code *avant* le commit (état parent), afin que les noms et plages de méthodes soient cohérents avec ce que le diff référence. Les résultats sont agrégés dans un fichier CSV contenant, pour chaque commit, les métriques individuelles. Les moyennes finales sont ensuite calculées sur l'ensemble des commits pour lesquels la vérité terrain est non vide.

6.5 Limites connues de la vérité terrain

La vérité terrain telle que définie ici est robuste, mais présente trois cas limites à prendre en compte lors de l'interprétation des résultats.

Méthodes nouvellement introduites par le diff. La couverture est collectée avant le commit, dans l'état du code parent. Si le commit introduit une méthode entièrement nouvelle, aucun scénario ne peut l'avoir couverte à ce stade : la vérité terrain ne peut pas identifier les scénarios qui, après le commit, invoqueront cette méthode. Ce cas tend à *sous-estimer* G et à pénaliser RTS-LLM s'il a correctement sélectionné ces scénarios.

Fichiers supprimés par le diff. Lorsqu'un commit supprime entièrement un fichier, les méthodes qu'il contenait ne figurent plus dans l'index de l'analyse statique. Elles ne sont donc pas incluses dans $MthodesModifies(diff)$, même si des scénarios les couvraient dans la version parente.

Surcharges de méthodes. Les identifiants de méthodes utilisés n'incluent pas la signature (types des paramètres). Si une classe possède plusieurs surcharges d'une même méthode et que seule l'une est modifiée, la vérité terrain peut inclure des scénarios qui n'utilisaient en réalité qu'une autre surcharge. Ce cas tend à *surestimer* légèrement G .

Ces trois limites sont inhérentes à toute approche de vérité terrain par couverture dynamique et sont documentées dans la littérature scientifique [13, 4]. Elles constituent des biais connus dont il faut tenir compte lors de l'analyse des résultats, sans remettre en cause la validité globale de la méthodologie.

7 Résultats et Benchmarks

Nous avons vu que la vérité terrain est l’élément central de toute évaluation d’un outil de sélection de tests : sans elle, il est impossible de savoir si l’outil sélectionne les bons scénarios ou s’il se trompe. Elle constitue la référence objective à laquelle toute sélection automatique doit être comparée.

Pour exploiter cette vérité terrain, il faut un **benchmark** : un protocole d’évaluation structuré qui soumet l’outil à un ensemble de situations réelles et mesure ses performances de manière reproductible. Le benchmark joue ici un rôle clé. Il ne s’agit pas simplement de vérifier que l’outil “fonctionne” dans quelques cas favorables, mais de le confronter à la diversité des commits réels, des modifications triviales aux refactorisations complexes, pour obtenir une image honnête de ce qu’il sait faire et de ce qu’il rate. C’est cette image qui permet de prendre des décisions éclairées sur l’utilisation de l’outil en production et sur les axes d’amélioration prioritaires.

Dans notre cas, le benchmark sert également à répondre à une question précise : l’ajout d’une analyse statique du code apporte-t-il réellement quelque chose par rapport au LLM utilisé seul ? Les résultats qui suivent quantifient cet apport commit par commit, sur les trois métriques standard : précision, rappel et F1.

7.1 Résultats quantitatifs

Le benchmark compare deux configurations de RTS-LLM sur 38 commits du projet **spring-cucumber-rest-api**, en utilisant le modèle Mistral 7B-instruct (q3_K_M) via Ollama sur GPU local (NVIDIA RTX A500, 4 Go VRAM).

- **LLM seul** : le diff et tous les scénarios BDD sont soumis directement au modèle, qui sélectionne seul les scénarios à retester.
- **Hybride** : un outil d’analyse de code identifie d’abord quels scénarios sont reliés aux méthodes modifiées en remontant les chaînes d’appel ; le LLM reçoit uniquement ces candidats et décide lesquels sont réellement impactés.

7.1.1 Métriques moyennes

Configuration gt > 0 (commits actifs) (tous commits) (tous commits)	Commits	Précision	Rappel	F1 moyen
LLM seul	24	0,487	0,368	0,227
Hybride (statique + LLM)	17	0,789	0,372	0,378
Gain	—	+0,302	+0,004	+0,151

TABLE 3 – Métriques moyennes sur les commits avec `gt_size > 0` (vérité terrain non vide). Modèle : Mistral 7B-instruct q3_K_M.

7.1.2 Distribution du rappel par commit

Tranche de rappel	LLM seul ($n = 24$)	Hybride ($n = 17$)
$R = 0$ (aucun scénario impacté détecté)	13 commits (54 %)	6 commits (35 %)
$0 < R < 0,5$ (détection partielle)	2 commits (8 %)	5 commits (29 %)
$0,5 \leq R < 1$ (bonne détection)	2 commits (8 %)	1 commit (6 %)
$R = 1$ (détection complète)	7 commits (29 %)	5 commits (29 %)

TABLE 4 – Distribution du rappel sur les commits avec `gt_size` > 0.

7.2 Interprétation des résultats

7.2.1 Pourquoi le mode hybride obtient une précision bien meilleure

Le gain en précision est le résultat le plus marquant du benchmark (+0,302, soit +62 % relatif). Il s'explique directement par le rôle de l'analyse statique comme pré-filtre.

En mode LLM seul, le modèle reçoit la totalité des scénarios BDD sans savoir lesquels exécutent réellement le code modifié. Il se base alors sur la *ressemblance de mots* entre le diff et les descriptions de scénarios : il sélectionne un scénario si sa description contient des termes proches du diff (noms de méthodes, type d'opération, champs modifiés). Ce raisonnement produit des **faux positifs** : des scénarios dont le texte ressemble au diff mais qui n'exécutent pas le code modifié. La précision de 0,487 signifie qu'en moyenne, sur deux scénarios sélectionnés par le LLM seul, un seul est réellement impacté.

En mode hybride, seuls les scénarios dont le code de test appelle (directement ou indirectement) une méthode modifiée sont transmis au LLM. Ce filtre préalable élimine les faux positifs par ressemblance de mots : un scénario ne peut figurer dans la liste que s'il est prouvé qu'il passe par le code modifié. Le LLM reçoit en plus le chemin d'appel reliant chaque scénario à la méthode modifiée, ce qui lui permet de confirmer ou d'écarter chaque cas avec des informations concrètes plutôt que de deviner. La précision de 0,789 signifie que près de quatre scénarios sélectionnés sur cinq sont réellement impactés.

En résumé : l'analyse de code sert de filtre objectif qui supprime les faux positifs que le LLM produirait par ressemblance de mots. Le LLM, muni du chemin d'appel concret, peut ensuite juger si chaque scénario est vraiment impacté par le changement.

7.2.2 Pourquoi le rappel n'augmente presque pas

Malgré la nette amélioration de précision, le rappel reste quasi identique (0,368 *vs* 0,372, +0,004). Ce résultat peut sembler paradoxal : si l'analyse statique « trouve » les bons scénarios, pourquoi ne pas mieux les sélectionner ?

Le LLM introduit ses propres faux négatifs. En mode hybride, si le modèle juge qu'un lien est trop indirect ou que la modification est mineure, il écarte ce scénario de la sélection finale — même si l'analyse de code l'avait correctement identifié comme impacté. Le gain apporté par le filtre de code est donc partiellement annulé.

L'analyse de code produit elle aussi des faux négatifs. Les méthodes ajoutées par le diff, les appels faits par réflexion ou par héritage, et les classes supprimées ne sont pas détectés par l'outil d'analyse. Si un scénario est impacté par l'un de ces cas, il ne figurera jamais dans la liste des candidats et sera manqué, quelle que soit la décision du LLM.

La baisse des sélections vides compense ces faux négatifs. Le seul gain de rappel visible est la baisse du taux de sélection vide (de 54 % à 35 %). Quand le LLM seul ne voit pas de lien évident, il ne sélectionne rien ; quand le mode hybride lui fournit un chemin d'appel concret, il sélectionne au moins quelques candidats. C'est cet effet qui explique les +0,004 de rappel moyen : les scénarios récupérés grâce au filtre compensent tout juste les faux négatifs introduits par le LLM.

Ce que révèle la distribution du rappel

La distribution (Tableau 4) donne une image plus fine que les seules moyennes.

Le mode LLM seul fonctionne en tout ou rien. 54 % des commits produisent $R = 0$ et 29 % produisent $R = 1$, avec très peu de cas intermédiaires (16 %). Soit le LLM ne voit aucun lien et ne sélectionne rien, soit il identifie un lien évident et sélectionne correctement tous les scénarios impactés. Les détections partielles sont rares.

Le mode hybride couvre davantage de cas intermédiaires. La proportion de commits avec $0 < R < 1$ passe de 16 % à 35 %. Le filtre de code fournit une liste de candidats que le LLM affine : cette sélection partielle vaut mieux que rien — même un rappel de 0,3 ou 0,5 réduit déjà fortement le nombre de tests à exécuter.

Le taux de détection complète ($R = 1$) est identique (29 %). Les commits sur lesquels l'outil trouve tous les scénarios impactés sont aussi nombreux dans les deux modes. Ce sont des commits avec un diff court ciblant une ou deux méthodes : le lien avec les scénarios est si évident que le LLM le détecte directement, sans avoir besoin du filtre de code.

Les 7 commits avec $R = 1$ en mode LLM seul partagent tous un diff court, ciblant une méthode dont le nom ou le rôle est suffisamment parlant (`ControllerAdvice`, `authenticate`, `getAll...`) pour que le LLM fasse le lien sans aide extérieure.

7.2.3 Lecture synthétique : deux profils de commits

L'ensemble des résultats converge vers un constat : les performances de l'outil dépendent fortement du **profil du commit**, indépendamment du mode utilisé.

Commits avec un diff court et ciblé. Le diff modifie une ou deux méthodes dont le rôle est précis et limité. Peu de scénarios passent par ce code, donc peu sont impactés. Le lien entre la modification et les scénarios concernés est clair, et les deux modes trouvent correctement tous les scénarios ($R = 1$). C'est le cas idéal pour RTS-LLM.

Commits avec un diff long touchant du code partagé par tous les scénarios. Le diff modifie du code exécuté par presque tous les scénarios (gestion des requêtes HTTP, authentification, gestionnaire d'erreurs global). Presque tous les scénarios sont donc impactés, mais l'outil n'ose pas tous les sélectionner. Le résultat est souvent une sélection vide (LLM seul) ou partielle (hybride), avec un rappel très faible.

Quand la modification touche du code partagé par tous les scénarios, l'outil ne peut pas remplacer l'exécution complète de la suite de tests. Il reste cependant utile pour la majorité des commits du quotidien, dont le diff est court et ciblé sur une fonctionnalité précise.

7.3 intégration CI/CD

RTS-LLM est conçu pour s'intégrer nativement dans un pipeline CI/CD sans modifier le reste du workflow existant. L'intégration repose sur l'option **-selection-only** de l'outil, qui produit en sortie une liste de noms de scénarios à la place du rapport complet.

Dans un pipeline CI/CD, cette étape s'insère entre le checkout du code et l'exécution des tests : RTS-LLM est appelé avec le commit courant et son parent comme bornes du diff, et sa sortie est transmise directement au runner Cucumber via un filtre sur les noms de scénarios.

Si la liste est vide, le pipeline bascule automatiquement sur l'exécution complète de la suite. L'outil s'intègre ainsi comme une étape standard dans GitHub Actions, Jenkins ou GitLab CI, sans modifier le reste du pipeline existant.

7.4 Intégration des API LLM distantes

Notre outil supporte nativement plusieurs fournisseurs de modèles, dont des API distantes comme OpenAI ou Anthropic. Cette flexibilité est importante : des modèles plus puissants qu'un Mistral 7B local (GPT-4, Claude, ...) pourraient significativement améliorer la précision et réduire les hallucinations observées dans nos benchmarks. Cependant, l'utilisation d'API distantes est freinée par le coût : chaque appel est facturé au token, et un benchmark sur 38 commits avec des prompts de 9,000 à 11,000 caractères représente une dépense non négligeable.

7.5 Perspectives d'amélioration

Les résultats identifient deux axes de progrès prioritaires.

Réduire le taux de sélection vide ($R = 0$). C'est le principal levier pour améliorer le rappel. La cause dominante en est la **taille réduite du modèle** : faute de ressources, l'évaluation a reposé sur Mistral 7B en quantification q3_K_M, dont la capacité de raisonnement reste limitée. Lorsque le diff est long ou le lien indirect, le modèle préfère ne rien sélectionner plutôt que de risquer une réponse. Cette limitation matérielle conditionne directement les pistes d'amélioration :

1. recourir à un modèle nettement plus grand (13B, 34B), moins réticent à sélectionner ;
2. augmenter la limite de taille du diff transmis (actuellement 4 000 caractères) pour fournir plus de contexte ;
3. reformuler le prompt du mode hybride afin d'éviter les réponses hors-sujet sur contexte partiel.

Améliorer la précision du mode LLM seul. Une précision de 0,487 signifie qu'un scénario sélectionné sur deux est un faux positif. Fournir au LLM quelques informations sur la structure du code, même sans analyse complète des chaînes d'appel, pourrait limiter les sélections basées uniquement sur la ressemblance de mots.

8 Gestion de projet

Notre TER s’est organisé en plusieurs phases successives, depuis la constitution de l’état de l’art jusqu’au développement des outils de sélection de tests de non-régression.

8.1 Déroulement du projet

Le diagramme ci-dessous retrace la chronologie des travaux réalisés de février à mai 2026.

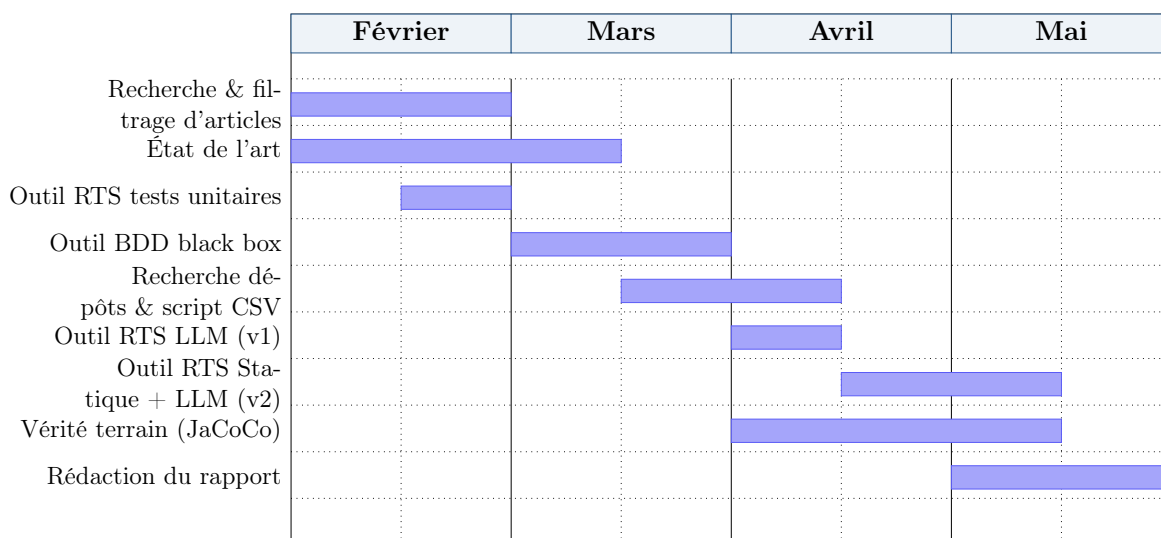


FIGURE 9 – Diagramme de Gantt du déroulement du TER

8.2 Organisation et gestion de projet

Dès le début du projet, nous avons adopté **JIRA** comme outil de gestion de tâches. Chaque article à lire, chaque fonctionnalité à implémenter et chaque livrable a été tracé sous forme de ticket, avec attribution à un membre de l’équipe et suivi de l’avancement. Cela nous a permis de maintenir une vision claire de l’avancement global du projet et de coordonner efficacement le travail à trois.

8.3 Réunions hebdomadaires avec l’encadrant

Tout au long du projet, nous avons effectué des réunions hebdomadaires avec notre encadrant, M. Abdelhak Djamel Seriai. Ces rendez-vous réguliers ont permis de suivre en continu l’avancement de nos travaux : présentation des résultats obtenus, discussion des difficultés rencontrées, réorientation des efforts si nécessaire. À chaque séance, on pouvait confirmer les décisions prises depuis la semaine dernière et fixer les objectifs de la semaine suivante, ce qui garantissait une évolution structurée et cohérente du projet.

8.4 Objectifs expérimentaux

En parallèle de l'état de l'art, des objectifs de validation expérimentale ont été définis. L'objectif idéal serait de réaliser un script qui exécute les tests fonctionnels d'un projet et remplit automatiquement un tableau de vérité terrain comportant :

- le hash du commit testé ;
- si le code est testable ;
- le nombre de scénarios / stories / features Gherkin ;
- si les fichiers Java ont été modifiés (oui/non) ;
- si les fichiers `.feature` ont été modifiés (oui/non).

Ces données permettraient d'évaluer objectivement la précision et le rappel de nos outils par rapport à la vérité terrain. Des pistes ont également été explorées pour identifier davantage de projets BDD open source exploitables comme sujets d'expérimentation.

9 Bilan

Ce TER portait sur la sélection de tests de non-régression dans les projets BDD, un sujet presque vierge, comme l’a montré notre état de l’art : sur quinze articles étudiés, un seul traite vraiment de la RTS appliquée au BDD. C’est ce vide qui a guidé notre travail.

Ce que nous avons réalisé

Avant tout développement, nous avons mené un état de l’art structuré selon un protocole proche d’un *Systematic Mapping Study*, qui a cartographié les approches existantes et révélé le vide sur lequel repose notre contribution. En parallèle des outils, nous avons aussi construit une méthodologie d’évaluation à part entière : une vérité terrain mesurée par couverture dynamique (JaCoCo), qui sert d’oracle objectif pour juger la qualité des sélections.

Côté développement, nous avons réalisé trois outils complémentaires. Le premier sélectionne les tests unitaires par analyse statique du code, posant les briques techniques de base : graphe de dépendances, analyse AST, lecture des diffs Git. Le deuxième, en boîte noire, descend au niveau du scénario Gherkin par similarité TF-IDF, sans accéder au code source. Le troisième, notre contribution principale, s’appuie sur les LLM pour relier les modifications du code aux scénarios concernés : une approche hybride mêlant filtrage statique sûr et raffinement par LLM, sans équivalent connu dans la littérature.

Ce qui marche, et ce qui reste à améliorer

Sur notre benchmark, l’approche hybride fait passer la précision de 0,49 à 0,79 sans dégrader le rappel. Le rappel reste toutefois le point faible, et la cause principale en est l’**hallucination** du modèle : trop petit (Mistral 7B quantifié), il génère des réponses incohérentes ou des sélections vides dès que le diff s’allonge ou que le lien est indirect. Tester un modèle nettement plus grand est donc la piste prioritaire pour fiabiliser ses réponses. Deux autres limites subsistent : les commits touchant du code partagé par tous les scénarios restent impossibles à réduire, et l’évaluation, menée sur un seul projet, demande à être élargie. L’outil n’en reste pas moins utile pour la majorité des commits courants, au diff court et ciblé.

Ce que ce projet nous a apporté

Ce TER nous a fait vivre un vrai travail de recherche, des premiers articles jusqu’à la validation expérimentale. Nous avons mesuré combien structurer un état de l’art, formaliser une idée puis l’évaluer honnêtement est bien plus que difficile, et aussi exigeant que le développement. Nous en sortons avec une meilleure compréhension du domaine des tests, une première expérience de l’intégration d’un LLM, et la pratique d’outils nouveaux pour nous (JGit, JaCoCo).

Le projet nous a aussi appris à coder à plusieurs sur une même base et à tenir un projet sur plusieurs mois.

Surtout, il nous a donné l’occasion rare, à ce stade de nos études, de contribuer à un

véritable problème industriel : la réduction du coût des tests en intégration continue est un enjeu concret, et savoir que notre approche y apporte une réponse, même perfectible, a été une vraie source de motivation.

Ouverture

Ce travail montre qu'on peut sélectionner les tests au niveau du scénario Gherkin, là où les outils actuels ne voient Cucumber que comme un bloc indissociable.

Alors que l'intégration continue se généralise et que les suites de tests grossissent, réduire le coût de la non-régression sans sacrifier la fiabilité répond à un besoin réel.

En explorant ce croisement encore neuf entre RTS, BDD et LLM, nous ouvrons une piste que d'autres pourront prolonger : tester des modèles plus puissants, réduire les sélections vides, intégrer les outils en CI/CD, ou les évaluer sur davantage de projets. Autant de pistes vers des outils plus robustes et réellement utilisables en production.

Références

- [1] Mohammed Abdalla Al-Refai and Mahmoud M. Hammad. A package level regression test selection approach for java software systems. *IEEE Access*, 13 :87848–87861, 2025.
- [2] Maral Azizi and Hyunsook Do. Retest : A cost effective test case selection technique for modern software development. In Sudipto Ghosh, Roberto Natella, Bojan Cukic, Robin S. Poston, and Nuno Laranjeiro, editors, *29th IEEE International Symposium on Software Reliability Engineering, ISSRE 2018, Memphis, TN, USA, October 15-18, 2018*, pages 144–154. IEEE Computer Society, 2018.
- [3] Israr Ghani, Wan Mohd Nasir Wan-Kadir, Adila Firdaus Bt Arbain, and Imran Ghani. A detection-based multi-objective test case selection algorithm to improve time and efficiency in regression testing. *IEEE Access*, 12 :114974–114994, 2024.
- [4] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*, pages 211–222, 2015.
- [5] Shunhui Ji, Bixin Li, and Pengcheng Zhang. Test case selection for all-uses criterion-based regression testing of composite service. volume 7, pages 174438–174464, 2019.
- [6] Owolabi Legunsen, Farah Hariri, August Shi, Yafeng Lu, Lingming Zhang, and Darko Marinov. An extensive study of static regression test selection in modern software evolution. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 583–594, 2016.
- [7] Yu Liu, Jiyang Zhang, Pengyu Nie, Milos Gligoric, and Owolabi Legunsen. More precise regression test selection via reasoning about semantics-modifying changes. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, pages 664–676, 2023.
- [8] Cláudio Magalhães, Flávia A. Barros, Alexandre Mota, and Eliot Maia. Automatic selection of test cases for regression testing. In *Proceedings of the 1st Brazilian Symposium on Systematic and Automated Software Testing, SAST 2016, Maringa, Parana, Brazil, September 19-20, 2016*, pages 8 :1–8 :8. ACM, 2016.
- [9] Willian D. F. Mendonça, Wesley K. G. Assunção, and Silvia R. Vergilio. Feature-oriented test case selection and prioritization during the evolution of highly-configurable systems. volume 217, page 112157, 2024.
- [10] Amir Ngah. Regression test selection by exclusion. 2012.
- [11] Rongqi Pan, Mojtaba Bagherzadeh, Taher Ahmed Ghaleb, and Lionel C. Briand. Test case selection and prioritization using machine learning : a systematic literature review. *Empir. Softw. Eng.*, 27(2) :29, 2022.
- [12] Lukas Rosenbauer, David Pätzelt, Anthony Stein, and Jörg Hähner. Correction : A learning classifier system for automated test case prioritization and selection. *SN Comput. Sci.*, 3(6) :462, 2022.
- [13] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2) :173–210, 1997.
- [14] Miriam Ugarte, Pablo Valle, Miren Illarramendi, and Aitor Arrieta. Enhancing multi-objective test case selection through the mutation operator. *Autom. Softw. Eng.*, 32(1) :18, 2025.

- [15] Jincheng Xu, Qingfeng Du, and Xiaojun Li. A requirement-based regression test selection technique in behavior-driven development. In *IEEE 45th Annual Computers, Software, and Applications Conference, COMPSAC 2021, Madrid, Spain, July 12-16, 2021*, pages 1303–1308. IEEE, 2021.
- [16] Lingming Zhang. Hybrid regression test selection. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 199–209, 2018.